Performance Optimization Strategies for Data Transmission From Edge to Cloud: A Review

Jian Liu, Yangyang Lin, Ziguang Fu, Gexi Lin, Guodao Sun, Zhu Xiao, *Senior Member, IEEE*, Yilong Zhang, *Member, IEEE*, Dongdong Zhao, Peng Chen, *Member, IEEE*, and Ronghua Liang, *Senior Member, IEEE*

(Survey Paper)

***Abstract*—With the rapid proliferation of IoT devices, the volume of generated data is growing at an unprecedented pace. Due to the limited resources of edge devices, a significant portion of this data must be transmitted to the cloud for in-depth processing, large-scale analysis, long-term storage, and archival purposes. Consequently, the performance has become a critical concern. While identifying prevailing challenges and research gaps in this domain requires a systematic review, such efforts remain largely absent from existing survey literature. This article addresses this gap by offering a structured review of recent optimization approaches. It begins by categorizing the literature into three main strategies: lossless transmission, lossy transmission, and hybrid approaches. In the context of lossless transmission, we analyze techniques such as data compression algorithms and incremental versus full synchronization mechanisms. For lossy strategies, we analyze approaches including lossy compression and predictive methods. In addition, we investigate hybrid strategies that integrate both lossless and lossy techniques to leverage their complementary advantages. Finally, we discuss the limitations of existing studies and highlight promising directions for future research in optimizing edge-to-cloud data transmission.**

***Index Terms*—IoT, data compression, data transmission, performance optimization.**

I. Introduction

NOWADAYS, IoT data is growing explosively, driven by the widespread deployment of IoT devices across many domains, such as smart grids [1], smart transportation [2], and smart cities [3]. As predicted by IDC, there will be 41.6 billion connected IoT devices generating 79.4 zettabytes of data by 2025 [4]. With the rapid advancement of IoT technology, this growth trajectory is expected to continue accelerating even further, creating both significant opportunities and challenges in the realms of Big Data management and analysis.

Given the resource-constrained nature of edge devices, which typically possess limited computational power and storage capacity, the massive volumes of data collected at the edge must be continuously transmitted to the cloud for in-depth processing, large-scale analysis, long-term storage, and permanent archiving. However, this edge-to-cloud transmission process is often costly and time-consuming, primarily due to the constrained network bandwidth commonly found in edge environments. To alleviate the network burden and enhance transmission efficiency, various data reduction techniques have been proposed as straight-forward, yet effective solutions [5], [6], [7], [8], [9], [10], [11], [12], [13], [14], [15], [16], [17], [18].

One of the most intuitive examples is the use of compression algorithms, which aim at reducing data size at the edge before transmission, in a lossless or lossy manner [19], [20], [21], [22]. For example, Lu et al. [19] introduce a resource-aware, adaptive compression strategy that combines both lossless and lossy compression algorithms, such as *gzip, bzip2, ZSTD, SZ, and WebP*, on resource-constrained edge devices to improve IoT data transfer efficiency. As an important complement to lossless compression, data deduplication has achieved its success in storage optimization through chunk/file-level redundancy elimination [23]. Recently, its value proposition has expanded to edge-to-cloud transmission, often referred to as data synchronization, as demonstrated by solutions like QuickSync [24], PandaSync [25], DSync [26], and FASTSync [27]. Additionally, data prediction, an edge-cloud collaborative approach, employs statistical models and machine learning to predict data points, thereby reducing the frequency of transmitting actual data. For instance, Liu et al. [28] propose an enhanced TimeGAN [29] prediction model to synthesize data points in the cloud, generating close approximations of the original data. Such a model can be efficiently trained and fine-tuned using real data sampled from the edge, ensuring high accuracy in the synthetic data generation process.

Despite the proliferation of such methods, there remains a notable gap in the literature—a systematic and comprehensive survey on this topic is still lacking. Many previous related studies focus on wireless sensor networks (WSN), an area that

Received 12 May 2025; revised 23 April 2026; accepted 9 May 2026. Date of publication 12 May 2026; date of current version 8 August 2026. This work was supported in part by the National Natural Science Foundation of China under Grant U24A20247, Grant 62432014, Grant 62202430, Grant 62422607, and Grant 62372411, and in part by the Natural Science Foundation of Zhejiang Province, China under Grants LY24F020018 and Grant LD24F020005. Recommended for acceptance by Yunjun Gao. *(Corresponding author: Ronghua Liang.)*

Jian Liu, Yangyang Lin, Ziguang Fu, Gexi Lin, Guodao Sun, Yilong Zhang, Dongdong Zhao, Peng Chen, and Ronghua Liang are with the College of Computer Science and Technology, Zhejiang University of Technology, Hangzhou 310023, China (e-mail: jliu83@zjut.edu.cn; 202003151116@zjut.edu.cn; f1633905943@gmail.com; ccccchildren@gmail.com; guodao@zjut.edu.cn; zhangyilong@zjut.edu.cn; zhaodd@zjut.edu.cn; chenpeng@zjut.edu.cn; rhliang@zjut.edu.cn).

Zhu Xiao is with the College of Computer Science and Electronic Engineering, Hunan University, Changsha 410082, China (e-mail: zhxiao@hnu.edu.cn).

Digital Object Identifier 10.1109/TKDE.2026.3692662

has already been extensively covered in earlier surveys [30], [31], [32], [33], [34], [35], [36], [37], [38], [39], [40], [41], [42]. Additionally, a number of studies have concentrated on improving data transmission efficiency through protocol-level optimizations [43], [44], [45], [46]. To bridge this knowledge gap, this paper exclusively examines works that are directly relevant to our research focus—edge-to-cloud data transmission—incorporating data reduction techniques, specifically data compression, data synchronization, and data prediction. By narrowing the investigation scope, we not only save valuable research effort but also gain a deeper understanding of the strengths and shortcomings within this domain, ultimately providing a clear roadmap for future research endeavors.

In addition, a critical motivation for conducting this survey also arises from several technical trends that we have observed:

Trend #1: The ever-widening gap makes edge-to-cloud transmission an inevitable choice. Although edge devices have steadily improved over time, significant gaps persist across nearly all dimensions—including CPU, memory, storage, network bandwidth, and energy—when compared with cloud infrastructures. This gap is further exacerbated by the explosive growth of IoT data: massive data volumes are overwhelming the resource-constrained edge devices, making local storage or processing infeasible. For example, Yang et al. [47] report that an intelligent connected vehicle can generate at least 10 TB of data per day.

Trend #2: Optimization tasks are becoming increasingly complex. As shown in Fig. 7, during the first two decades (1996-2015), only a small fraction of studies (11.1%) addressed edge-to-cloud transmission, and these early efforts primarily focused on file-based data synchronization with a narrow goal of improving bandwidth utilization [48], [49], [50]. In contrast, the most recent decade (2016-2026) has witnessed an explosive growth in scholarly and industrial interest (88.9%). This period has also brought a dramatic diversification of application scenarios [21], [22], [51], [52], [53], [54], [55], introducing multi-dimensional heterogeneity across data modalities, edge devices, and optimization objectives, as shown in Table I, II. Consequently, strategies designed for earlier, simpler conditions are no longer sufficient; modern systems increasingly require context-, resource-, and task-aware solutions [21], [56], [57].

Trend #3: The rapid advancement of artificial intelligence (AI) is reshaping the transmission paradigm. On the edge side, AI-driven methods endowed with perception, prediction, and decision-making capabilities are replacing traditional heuristic-based techniques [24], [58], [59], offering a powerful mechanism for improved adaptivity. Representative systems, such as LearnedSync [60] and AdaEdge [57], demonstrate that learning-based controllers can substantially improve efficiency under highly dynamic network, workload, or hardware conditions. On the cloud side, AI also enables a fundamentally different transmission paradigm: instead of frequently transmitting raw data, predictive models can reconstruct high-fidelity approximations, thereby opening a new direction toward "model-driven" transmission [29], [52], [53]. Furthermore, the rapid development of large language models (LLMs) is expected to bring new opportunities and challenges for this field.

To systematically identify and analyze research on optimization strategies for edge-to-cloud data transmission, we begin with an unstructured search using Google Scholar to obtain a preliminary overview of relevant literature. In parallel, we conduct targeted searches for pertinent publications in leading journals and conferences within the IoT domain—such as the IEEE Sensors Journal, IEEE Internet of Things Journal, TOSN, MobiCom, TMC, World Forum on IoT, and HotEdge—as well as in major venues from the database and systems fields, including ICDE, FAST, USENIX ATC, TPDS, ICDCS, and TOS. During this process, we use a set of keywords related to transmission optimization (e.g., "IoT," "data transmission," "edge," "cloud," "performance") to identify an initial pool of relevant studies. Building upon this foundation, we further expand our collection by tracing both citations and references and by exploring the academic homepages of leading researchers in this field.

To ensure the relevance and quality of the selected studies, we apply the following exclusion criteria: 1) the keywords related to transmission optimization appear only in peripheral sections of the paper (e.g., "future prospects" or "related work"); 2) the research, although situated within the IoT domain, does not focus on optimizing edge-to-cloud transmission performance (e.g., works centered on WSN field); 3) the studies focus exclusively on protocol-level optimization rather than higher-level strategic transmission policies.

To guide this survey, we highlight three central research questions:

- What are the fundamental differences among existing techniques, and what are the respective advantages and limitations of each?
- How can we identify the most appropriate optimization strategy tailored to specific application scenarios?
- How can the rapid advancement of AI be effectively leveraged to enhance the transmission efficiency, and what new possibilities do they unlock?

The remainder of this paper is organized as follows: Section II reviews prior survey research on data transmission optimization within IoT scenarios. Section III presents the categorization framework and evaluation metrics used in this review. Section IV discusses lossless transmission strategies, including data transmission based on lossless compression, incremental synchronization, and full synchronization. Section V explores lossy transmission methods, covering lossy compression and predictive transmission approaches. Section VI delves into hybrid strategies that integrate lossy and lossless transmission techniques. Section VII introduces a unified optimization model for edge-to-cloud transmission. Section VIII discusses current research limitations, and Section IX highlights promising future directions. Finally, Section X concludes this paper.

II. Related Work

In this section, we review prior research on data transmission optimization within IoT scenarios, analyze their thematic scope and methodological focus, and highlight the principal distinctions between existing studies and our own work.

A significant body of survey literature has focused on data reduction techniques within WSN environments. Dias et al. [30]

TABLE I
CATEGORIZATION OF TRANSMISSION METHODS

Classification		Ref.	Data Types	Edge Devices	Compression Methods	Open Source
Lossless Transmission	Lossless Compression	[61]	Geospatial Data	Intel Edison	.zip, .tar.gz, .gzip, .tar, .iso, .zipx	-
		[62]	UCR, PAMAP, MSRC-12, UCI Gas, AMPDs (Time Series)	Resource-constrained Embedded Device	Sprintz	✓
		[63]	Traffic Information (Time Series)	MEC Server	MLOC, Simple Greedy, Hsadelta	-
		[64]	EEG Data (Time Series)	Fog Smart Gateway	Huffman + Agglomerative Clustering	-
		[65]	Mobile Web Traffic Data, Activity Recognition Data, Air Quality Data, RToF Measurements Data	Samsung Galaxy S5	IoTZip(Linear Regression)	-
		[66]	Vibration data (Time Series)	IoT-gateway	gzip, bzip2	-
		[67]	IoT sensor data (Time Series)	IoT-gateway	Optimal approach, UBA (daily/weekly model)	-
	Incremental Synchronization	[48]	File Data (Linux Kernel, Samba)	General-purpose Computers	Rsync	✓
		[49]	File Data (MS Word, GCC, Perl)	General-purpose Computers	LBFS	-
		[50]	File Data (MS Word, Binaries, Logs)	Ubuntu 12.04, OS X Lion 10.7	UDS (Update-Batch Delay Sync)	-
		[24]	File Data (MS Word, Binaries, Logs, Real Data)	Galaxy Nexus, Linux Laptop, EC2 Server, Linux Server	Network-aware Chunker, Redundancy Eliminator, Batched Syncer	-
		[68]	File Data (MS Word, SQLite, Binaries, Real Data)	EC2 m4.xlarge, Galaxy Note3	DeltaCFS	✓
		[69]	File Data (MS Word, Binaries, Real Data, Linux Kernel)	PC, Mobile Devices	WebR2sync+	✓
		[26]	File Data (Benchmark, Real-world)	General-purpose Computers	Dsync	✓
		[70]	File Data (NY Taxi, OS Snapshot)	General-purpose Computers	SimpleSync	-
		[71]	File Data (Linux Kernel, OSS Branch)	General-purpose Computers	FeatureSync (Feature-based Encrypted Sync)	✓
		[72]	File Data (Real-world, Linux Kernel, Large Files)	General-purpose Computers	WASMrsync+	✓
		[73]	Silesia, PPT, GLib, Pictures, Mails	PC, iPhone 11	Netsync (Deflate, LZ4)	-
		[27]	File Data (OSS Source Code, Encrypted)	Intel i5-6600, 32 cores, 95 GB memory	FASTSync	✓
		[74]	Big Spatiotemporal Data (Vehicle Trajectory, Smart Sensor, Urban Map, Satellite, Mobile Device)	-	SPsync	-
		[75]	Mobile Backup Files	Smartphones (Resource-limited)	File change detection based on operation logs	-
		[76]	General Files (Kernel/Wiki)	Multi-core Edge Servers	Fine-grained Parallel Chunking (CDC) + Streaming Matching Protocol	✓
		[77]	System Images / Source Code	SGX Encrypted Enclaves	Deduplication + Bidirectional Delta + Local Compression	✓
	Full Synchronization	[25]	File Data (File Sharing, Enterprise, Cloud Storage)	-	PandaSync	✓
		[78]	File Data (General, OSS Project, Real-world)	-	State Encoding, Push-pull Paradigm	✓
		[60]	File Data (General, Real-world)	-	LearnedSync	-
Lossy Transmission	Lossy Compression	[79]	Time Series (Highway Ramp, Household Smart Meter, Robot Sonar)	Lightweight Gateway, Standard Gateway, Commodity PC	NECTAR Agent	-
		[58]	GPS Data	Linux-based In-vehicle Platform	GIS Context-Aware Compression	-
		[51]	Vehicle Trajectory (GPS, Acceleration)	Android Platform (Smartphone)	Online Trajectory Compression Algorithm	✓
		[54]	Three-phase Current Signal (Photovoltaic Inverters)	NVIDIA Jetson Xavier NX	CS-CNN (CS + CNN)	-
		[55]	Visual Image Data	Resource-Constrained IoT Devices	Split-Compression	-
		[80]	Time Series (e.g., EEG)	Resource-constrained Microcontrollers (ESP32)	Lightweight Error-bounded Lossy Compression	✓
	Data Prediction	[59]	Temperature, Humidity, Light, Voltage Sensor Data	Star Network Topology	Cost-Aware Dual Prediction Scheme	-
		[28]	IoT Time Series Data	Raspberry Pi 4	Xender (Sampling + Generation)	-
		[52]	Vibration data (Time Series)	Edge gateway (Raspberry Pi 4B), Cloud server	LSTM-based prediction model (DPTR scheme)	-
		[81]	Vibration data (High-frequency Time Series)	Edge gateway, Cloud server	Informer-based long-sequence prediction model	-
		[53]	Industrial Vibration Data	Raspberry Pi 4B	DPS (Deep Learning Dual Prediction + Cloud-Edge)	-
		[82]	Time Series (e.g., ECG)	Fog Nodes	Kalman Filter Prediction + Redundant Data Filtering	-
		[83]	Smart Agriculture Data (Temp/Humidity)	Edge Stations	Lightweight LSTM Prediction Model	-

TABLE I
(CONTINUED.)

Classification	Ref.	Data Types	Edge Devices	Compression Methods	Open Source
Lossless And Lossless Combined Transmission	[19]	Time Series Data(Multi-domain sensor data)	IoT Devices	gzip, bzip2, lzma, ZSTD, SZ, WebP, GDCMConv, Resource-aware Adaptive	✓
	[21]	EEG Data (Time Series)	PDA, Raspberry Pi	TD-FE, FD-FE, DWT Compression	-
	[84]	IntelLab Dataset and MIT-BIH Arrhythmia Database.	Edge Nodes in Sensor Networks	LZ4, DFan	-
	[56]	ECG Data, Rovio Data, Sensor Data, Stock and Stock-Key Data, Micro Data	Banana Pi Zero M2, RockPi 4A, UP Board	LEB128-NUQ, ADPCM, UANUQ, UAADPCM, LEB128, Delta-LEB128, Tcomp32, Tdic32, RLE, PLA	-
	[57]	Time-series data (IoT sensor data)	Resource-constrained edge devices	BUFF, PAA, FFT, PLA, RRD-sample (lossy/lossless)	-
	[85]	Health Monitoring Data	Acquiring Stations	SDT + Huffman/LZW	✓

Classification: refers to category of information being evaluated; Reference: denotes the source of the quotation or reference; Data Type : refers to the specific data used for evaluation; Edge Device : refers to hardware platform used for tests, and - means not mentioned; Compression Method : describes the method applied; Open Source: ✓means the source code is publicly available, and - means not readily accessible.

TABLE II
METRICS FOR EVALUATING TRANSMISSION EFFICIENCY

Paper	**Compression Ratio**	**Throughput**	**Latency**	**Transmission Time**	**Bandwidth Utilization**	**CPU Utilization**	**Energy Consumption**	**Memory Consumption**
[61]		✓✓	✓✓					
[62]	✓✓✓		✓✓				✓	✓✓✓
[63]	✓✓✓							
[64]	✓✓✓				✓✓			
[65]		✓✓	✓				✓	
[66]	✓✓							
[67]	✓✓				✓		✓	
[48]	✓✓✓		✓		✓			
[49]					✓✓✓			
[50]			✓		✓✓✓	✓		
[24]				✓✓✓	✓			
[68]	✓					✓✓✓		
[69]		✓		✓✓		✓		
[26]		✓		✓✓✓				
[70]				✓✓	✓✓	✓✓		
[71]				✓✓✓	✓✓✓			
[72]		✓		✓✓		✓		✓✓✓
[73]		✓		✓✓✓		✓		
[27]				✓✓✓				
[74]				✓✓✓		✓✓✓		
[75]	✓✓			✓✓	✓	✓✓✓	✓	
[76]	✓	✓✓✓		✓✓		✓✓		
[77]	✓✓✓	✓✓			✓✓	✓		✓
[25]				✓✓✓	✓	✓		
[78]							✓	✓
[60]				✓✓✓	✓✓	✓		
[79]	✓✓		✓					
[58]	✓✓✓							
[51]	✓✓✓		✓✓				✓	✓
[54]	✓✓✓			✓				
[55]	✓✓✓					✓		
[80]	✓✓✓			✓		✓✓	✓✓✓	✓✓
[59]	✓✓✓						✓	
[28]		✓✓✓		✓✓✓	✓✓✓			
[52]	✓✓✓				✓			
[81]	✓✓	✓✓						
[53]	✓✓✓				✓			
[82]	✓✓✓		✓✓		✓		✓	
[83]	✓✓✓		✓		✓✓		✓	
[19]	✓✓				✓✓	✓✓		
[21]	✓✓		✓✓✓				✓✓	
[84]	✓✓							
[56]	✓✓	✓✓✓	✓✓✓				✓✓	
[57]	✓✓	✓✓			✓✓	✓✓		✓✓
[85]	✓✓		✓✓		✓			

In this table, multiple ✓symbols denote increasing levels of optimization; a single ✓denotes a metric is considered but not explicitly optimized, while blank entries denote metrics are not considered.

reviewed prediction-based data reduction methods in WSNs, categorizing them into single and dual prediction schemes. Ketshabetswe et al. [31] reviewed and compared data compression algorithms to address energy efficiency challenges in WSNs environments. Harb et al. [36] provided a detailed summary of data aggregation techniques implemented at both the sensor node and cluster-head levels in cluster-based WSNs. Srisooksai et al. [37] surveyed practical data compression strategies, focusing on both local and distributed algorithms, with particular emphasis on energy efficiency and applicability in real-world WSN deployments. Razzaque et al. [38] analyzed a range of compression methods for resource-constrained WSNs, highlighting predictive coding, distributed source coding, transform-based methods, aggregation, and compressed sensing as key approaches. Singh et al. [39] explored in-network data processing techniques grounded in compressed sensing, aiming to conserve energy in WSN applications. Lungisani et al. [40] conducted a comparative survey on image compression techniques in WSNs, emphasizing the need for energy-efficient solutions that balance compression ratio with reconstructed data quality. Jain et al. [41] examined data reduction techniques in the field of WSNs, including data filtering, data compression, data aggregation, and data prediction, proposing a taxonomy for minimizing cloud reliance and associated costs. Khan et al. [42] reviewed data reduction techniques focusing on predictive approaches within WSNs; however, their analysis was limited in scope, covering only 23 research articles. Correa et al. [34] developed a foundational taxonomy of lossy compression for IoT sensors, though the majority of studies reviewed were within WSN environments.

Beyond WSN-focused efforts, other surveys have examined transmission optimization from a protocol perspective. Kreković et al. [43] investigated communication protocol optimizations across the IoT-edge-cloud continuum, with peripheral treatment of data reduction techniques within WSNs. Xie et al. [44] analyzed IoT adoption in Canadian smart cities, placing emphasis on wireless communication standards and protocol design. Lo et al. [45] reviewed routing protocols in Delay-Tolerant Networks (DTNs), emphasizing strategies such as controlled message replication, contact probability estimation, and single-copy path optimization to minimize redundant transmissions. Agarkar et al. [46] presented an extensive review of routing protocols and challenges in WSNs, offering multidimensional technical comparisons to reduce energy consumption and extend network lifespan.

Recent works have also explored the integration of artificial intelligence (AI) into transmission optimization frameworks. Pioli et al. [86] conducted a systematic literature review on AI-driven data reduction techniques, with particular emphasis on leveraging the inherent strengths of AI paradigms, such as autoencoders, feature selection, and filtering, to enhance efficiency of IoT systems. Alwash et al. [87] concentrated on the Internet of Medical Things (IoMT), surveying data reduction methods such as quality-aware recording, compression, and feature extraction, albeit with a narrow focus on EEG data.

Although prior literature has thoroughly explored compression, prediction, and aggregation techniques for transmission optimization, much of this work remains bounded within WSN-centric domains or narrowly scoped around specific strategies, such as lossy compression [34], prediction-based reduction [30], or compressed sensing [39], as well as protocol-layer optimizations [43], [44], [45], [46] and AI-driven approaches [86], [87].

III. Categorization and Evaluation Metrics

Unlike conventional network transmission, where system resources are relatively abundant and optimization objectives can often be addressed independently, edge-to-cloud data transmission is fundamentally constrained by limited resources at the edge. In such settings, network bandwidth is typically the primary bottleneck, and reducing data volume to alleviate bandwidth pressure is a central objective; however, it cannot be pursued in isolation.

This is because edge devices are constrained not only in bandwidth, but across nearly all dimensions, including computational power, memory capacity, and energy availability. More importantly, these resources are highly dynamic in practical edge environments. As such, a fixed, resource-unaware design often induces bottleneck shifting. For example, aggressive compression can significantly reduce data volume but often introduces substantial computational overhead, which may saturate limited CPU capacity. *These coupled constraints, together with the resulting bottleneck-shifting effects, constitute a fundamental distinction from conventional settings, as they transform the problem from a single-objective optimization into a multi-objective trade-off, making it a key challenge in this field.*

Additionally, the relative importance of optimization objectives varies significantly across scenarios. For instance, latency-sensitive applications prioritize rapid processing, whereas energy-constrained edge sensing favors lightweight computation. Consequently, edge-to-cloud data transmission does not inherently prioritize a single, fixed metric, such as compression ratio, throughput, or latency, but instead seeks to dynamically balance these competing objectives, according to specific system conditions and downstream task requirements.

A. Categorization

To better understand this complex landscape, we organize the existing literature into a structured taxonomy based on data fidelity, categorized into lossless, lossy, and hybrid approaches, as shown in Table I. Notably, lossless transmission extends beyond conventional lossless compression to include data synchronization techniques. The former produces self-contained representations that can be independently decompressed for exact bit-level recovery. In contrast, data synchronization methods, such as incremental synchronization based on Content-Defined Chunking (CDC), transmit only differential updates and reuse unmodified chunks stored in the cloud to reconstruct the target file. Consequently, intermediate representations (e.g., CDC-generated chunks) may not preserve bit-level equivalence with the original data, however, the final reconstructed file maintains strict file-level semantic consistency. This distinction highlights that losslessness can be defined at different levels (bit-level vs. file-level), and our framework adopts the latter to faithfully reflect the operational logic of synchronization-based systems.

We further summarize the key dimensions of each study, which represent common and critical considerations in edge-to-cloud transmission. A detailed analysis of each study is presented in the subsequent sections.

Data type fundamentally determines which compression techniques are appropriate, as different data exhibit distinct semantic structures and redundancy patterns. For example, file-based data—including Linux kernel repositories, Microsoft Word documents, binary executables—are commonly used in data synchronization scenarios, where high cross-version redundancy makes them particularly well suited for deduplication. In contrast, time-series data, which appear in more than 30% of the studies surveyed, also contain substantial redundancy; however, their fine-grained variability in floating-point values makes it difficult to form large, easily deduplicated chunks, thus limiting the effectiveness.

Edge devices exhibit substantial diversity, with approximately 30 different types of devices presented, ranging from resource-constrained platforms such as Raspberry Pi to smartphones, unnamed IoT devices, gateways, and PCs. This hardware heterogeneity directly influences both the feasibility and the realized performance of proposed methods. For example, CStream [56] relies on asymmetry-aware scheduling on heterogeneous processors (e.g., RK3399); on non-heterogeneous platforms, however, the effectiveness is diminished dramatically. This observation further implies that transmission strategies must be carefully tailored, as achieving transferability across platforms with diverse capabilities remains challenging.

Compression methods refer to the specific algorithms employed to enhance transmission efficiency, with their adoption primarily determined by different data types. Classical algorithms such as gzip, bzip2, LZW, and ZSTD are predominantly applied to IoT sensor data, image data, etc. File-based data typically relies on synchronization-based methods, while predictive approaches, which leverage machine learning models such as LSTM and CNN, are mainly used for time-series data. Comparing these methods provides valuable insights into their technical strategies and intended application scenarios.

Open source is also taken into account, as it directly affects whether solutions can be easily reproduced, verified, and practically deployed, thereby encouraging subsequent research to build upon them. Notably, more than 30% of the surveyed works have made their source code publicly available. This reflects a growing trend toward open research practices.

B. Evaluation Metrics

Furthermore, to illustrate how different approaches navigate these multi-objective trade-offs, we summarize the reported metrics of each study in Table II. The results reveal a clear pattern of partial optimization, where individual objectives are typically considered in isolation rather than within a unified multi-objective framework.

The compression ratio, defined as the ratio of raw data size to compressed data size (or its inverse), is the most widely used metric, appearing in over 62% of the reviewed studies. While effective at quantifying data reduction, these approaches generally treat compression in isolation, without jointly modeling the computational overhead or memory footprint required to achieve such gains.

Beyond data reduction, system-level metrics such as throughput, transmission time, and bandwidth utilization are also widely reported (over 55%) as indicators of overall transmission efficiency. Despite their prevalence, these metrics—along with latency, which is critical for real-time applications [21], [65], [69]—are often optimized independently. This fragmented focus is further reflected in the limited consideration of hardware constraints, as only a small subset of studies (over 26%) incorporate CPU, memory, and energy factors into their optimization objectives.

We further provide a quantitative analysis of optimization priorities in Fig. 1. The lossless strategy primarily emphasizes overall transmission efficiency, exhibiting the highest median and the widest distribution in transmission time, alongside consistent optimization of compression ratio and bandwidth utilization. In contrast, the lossy strategy shifts its focus toward latency-sensitive objectives; it emphasizes compression ratio while balancing latency and energy consumption, reflecting scenarios where data fidelity is sacrificed to achieve aggressive data reduction and minimal delay. The hybrid strategy achieves a more balanced optimization profile, maintaining a high priority for compression ratio, latency, and bandwidth utilization, while also accounting for throughput, CPU utilization and energy consumption. Collectively, these observations reveal that existing approaches optimize objectives in isolation rather than within a unified framework, and that comprehensive resource utilization remains insufficiently explored.

IV. Lossless Transmission

Lossless transmission is designed to ensure that no information is lost during data transfer. This is particularly crucial for applications with stringent accuracy requirements. While directly transmitting raw data to the cloud can meet these demands, integrating data reduction techniques (e.g., lossless compression) can significantly enhance transmission efficiency.

We examine lossless transmission from two perspectives: lossless compression-based transmission and deduplication-based synchronization. For the former, the studies we reviewed either propose novel lossless compression algorithms [62], [63] or develop transmission strategies based on existing compression methods [61], [64]. For research focused on novel lossless compression algorithms, we will delve into the specific details such as Sprintz [62]. In contrast, given that conventional techniques like Huffman coding, LZW, and Gzip are well-established and have been widely used for years, our focus shifts to the optimized transmission strategies built upon them. For deduplication-based synchronization, which operates on coarse-grained data chunks, superior compression ratios and performance are achieved, particularly when synchronizing modified data files across different versions.

A. Data Transmission Based on Lossless Compression

Early studies directly adopt general-purpose compression algorithms to verify the feasibility of edge compression.

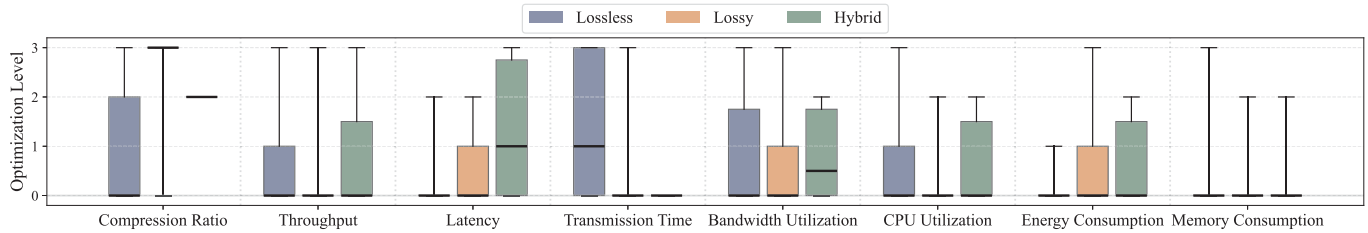


Fig. 1. An illustration of the optimization priorities for lossless, lossy, and hybrid transmission strategies, with levels quantified on a scale from 0 to 3 corresponding to the number of ✓entries in Table II.

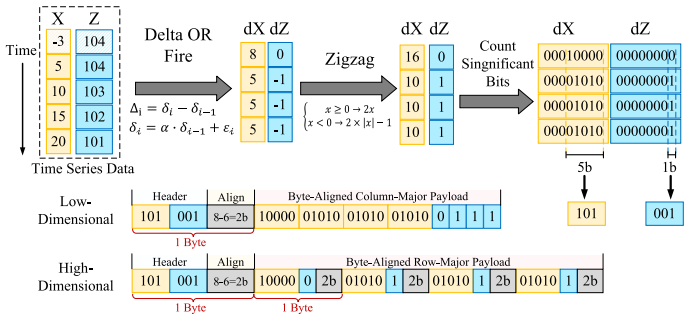


Fig. 2. Overview of Sprintz algorithm. Specifically, it first computes the differences between consecutive data points using delta encoding. These differences are then converted into non-negative integers via ZigZag encoding. Next, it determines the number of bits required to represent each value and packages them in a byte-aligned format. Finally, the header information is appended to describe how data is encoded and organized.

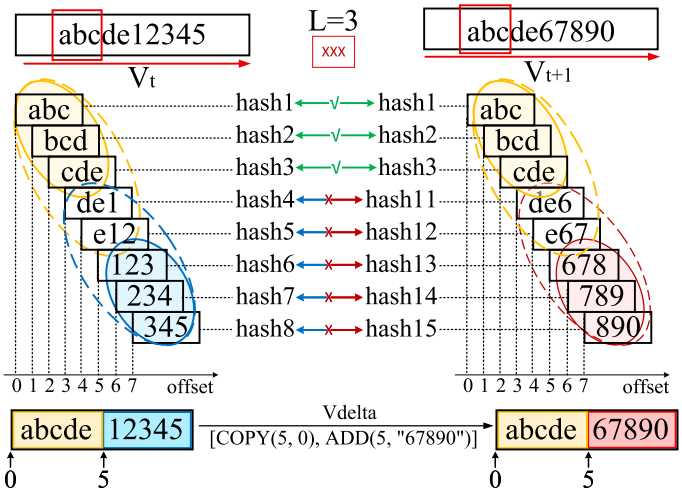


Fig. 3. Overview of MLOC algorithm. Identify non contiguous but reusable short fragments through the "hash extension merge" strategy. Delta compression between two consecutive versions of vehicle data (V_t: "abcde12345" and V_{t+1}: "abcde67890").

Barik et al. [61] proposed the *FogGIS* framework to mitigate transmission delays associated with geospatial data by leveraging fog computing. In this architecture, the compression task is delegated to the fog layer, which is located closer to the data source, thereby enabling early-stage data reduction before uploading to cloud. FogGIS incorporates a suite of lossless compression algorithms, e.g.,. tar.gz,. zip,. tar,. iso, to minimize data volume. Experimental results demonstrate that the. tar.gz algorithm outperforms others in most cases. By substantially decreasing the amount of data transmitted to the cloud, FogGIS improves end-to-end transmission efficiency.

Idrees et al. [64] proposed a lossless compression method for compressing electroencephalogram (EEG) data at fog computing layer before uploading it to the cloud. The method integrates hierarchical clustering with Huffman coding. In the clustering step, EEG data points are grouped into similar clusters using a bottom-up approach, with Euclidean distance metrics guiding the process. Huffman coding is subsequently applied to each cluster to achieve more efficient compression. This technique effectively reduces the amount of data that needs to be transmitted without compromising data quality.

However, domain-specific compression algorithms are generally more efficient for specific data types, thus delivering additional benefits.

To reduce the high transmission overhead associated with IoT data, **Blalock et al.** [62] proposed *Sprintz*, a lossless compression framework specifically optimized for time-series data on resource-constrained IoT devices, as illustrated in Fig. 2. Sprintz employs two types of predictors: Delta encoding and the FIRE predictor. Delta encoding computes the difference between consecutive values, offering high computational efficiency suitable for memory-constrained edge devices. In contrast, the FIRE predictor leverages online gradient descent to dynamically refine prediction coefficients, substantially minimizing prediction residuals. Once the raw sensor stream is converted into a residual sequence, Sprintz applies a multi-stage encoding pipeline. First, the ZigZag transformation reduces sign-bit overhead by mapping signed integers to compact unsigned values. Subsequently, adaptive bit packing allocates variable-length slots based on the magnitude of residuals, optimizing space utilization. Run-length encoding further compresses sequences of zero-error blocks, which are common in sampled sensor data. Finally, Huffman coding applies entropy reduction by assigning shorter codes to frequent residual values.

Similarly, in Internet of Vehicles (IoV) scenarios, conventional delta compression techniques (e.g., Simple Greedy [88], Hsadelta [89]) reduce redundant data transmission by encoding incremental changes. However, their over-reliance on identifying the longest matches often overlooks shorter, yet more efficient reusable segments. To address this limitation, **Hu et al.** [63] proposed *Maximizing Length and Optimizing Copy operations (MLOC)*, a novel delta compression algorithm designed for optimal redundancy elimination. As illustrated in Fig. 3, MLOC operates in a two-stage process. First, the old version V_t

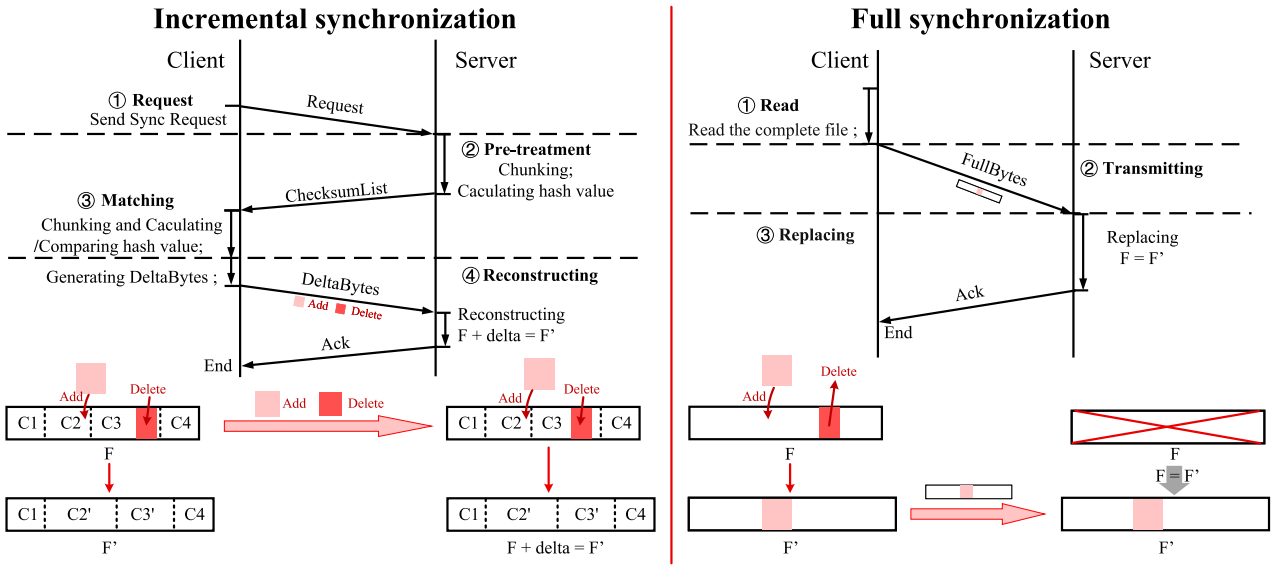


Fig. 4. The comparison between incremental synchronization and full synchronization strategies. Incremental synchronization only transmits differences through block hashing but has high computational overhead, while full synchronization is simple but may transmit redundant data, revealing the fundamental trade-off between computation and transmission.

is divided into overlapping 3-byte segments, such as "abc" and "bcd", generating a sequence of hash values (hash1-hash8) with corresponding offsets (0-9). The new version V_{t+1} undergoes the same hashing process to identify potential matches. Second, it expands and merges these matches backward and forward using a dynamic window sliding approach. For instance, an initially detected match "abc" extends to "abcd" and further to "abcde". The longest reusable prefix, "abcde," is encoded as *COPY*(5,0), while unmatched segments ("67890" in V_{t+1}) are encoded as *ADD*(5,"67890").

Further research recognizes that compression does not operate in isolation, but is influenced by dynamic environments.

Existing compression techniques often struggle to handle the heterogeneity of IoT data and fluctuating network conditions, failing to achieve an optimal balance between compression efficiency and computational overhead. To resolve this, **Melissaris et al.** [65] propose *IoTZip*, a lightweight library designed for intelligent selective compression at the edge. IoTZip dynamically assesses compression decisions based on data characteristics (e.g., type, size, compressibility predicted via regression models) and real-time network throughput estimates. It employs a threshold-based policy to exclude small or non-compressible data and utilizes a cost-benefit formula to determine whether compression is advantageous, ensuring that the total latency (i.e., compression time + compressed transfer time) remains lower than the uncompressed transfer time.

Most existing works focus primarily on sensor-level data reduction while overlooking user access patterns, thereby failing to fully exploit compression and scheduling opportunities at the gateway. To address this limitation, **Ng et al.** [66], [67] propose a context-aware data reduction framework for IoT systems that explicitly incorporates user behavior into transmission decisions. Specifically, the framework dynamically determines optimal data upload delays by analyzing user behavior. For users accessing data at fixed intervals (e.g., every 5 minutes), the IoT-gateway leverages the Nyquist-Shannon theorem to delay uploads by half the user's access interval (e.g., 2.5 minutes), ensuring data arrives just before the next access. Conversely, for non-fixed access patterns, the system identifies peak and off-peak periods and computes the maximum allowable delay during off-peak hours using the equation: $\mathrm{D} = I_{min} - P - T$, where I_{min} represents the shortest observed access interval, P denotes compression time, and T accounts for transmission time.

B. Data Synchronization

1) Incremental Synchronization: It is a technique that transmits only the modified portions of data, thereby substantially reducing network traffic. As depicted in the left portion of Fig. 4, the process begins with the client issuing a synchronization request. Upon receiving the request, the server divides the target file into multiple chunks and computes their corresponding fingerprints using cryptographic hash functions such as SHA1 or MD5. The resulting list of hash digests is then sent back to the client. On the client side, a similar procedure is performed: the file is segmented and hashed, and the locally computed hash values are compared against the server-provided checksum list to identify the modified chunks. These updated chunks are subsequently transmitted to the server, which reconstructs the file accordingly. The lower-left portion of the figure further illustrates this client-server interaction and the reconstruction of the file based on the incremental updates. Depending on the chunking strategy employed, incremental synchronization approaches can be broadly categorized into two groups. The first group adopts fixed-size chunking, wherein files are divided into blocks of uniform length, as demonstrated in [48], [50], [68], [69]. In contrast, the second group utilizes content-defined

chunking, where the block boundaries are determined based on the file content itself, as exemplified by [24], [26], [49], [90].

Fixed-size Chunking (FSC) is a straightforward technique where data is divided into chunks of a predefined, constant size, regardless of the content. Due to its simplicity, FSC is generally faster and easier to implement, making it well-suited for scenarios where data changes infrequently and uniform chunk sizes are advantageous. However, a major drawback of this approach lies in its susceptibility to the "boundary shifting" problem. In the following sections, we present a comprehensive review of existing incremental synchronization techniques that adopt the FSC approach.

Tridgell et al. [48] introduced the "*rsync*" application, which employs the fixed-block method for data synchronization. The process begins when the sender initiates a synchronization request to the receiver. Upon receiving the request, the receiver divides the target file into fixed-length blocks, calculates the checksum for each block, and generates a checksum list, which is then sent back to the sender. Each checksum includes a weak hash for efficiency and a strong hash to minimize collision probability. Upon receiving the checksum list, the sender computes the hash values of its own file blocks and compares them with those in the receiver's list. Based on the comparison, delta bytes are generated for the unmatched blocks and sent to the receiver for reconstructing the modified file.

Although incremental synchronization (e.g., rsync) has been widely deployed in PC clients and mobile applications, it faces significant challenges in efficiently supporting web browsers, which serve as a cross-platform and installation-free access method. **Xiao et al.** [69] introduced the design and implementation of *WebRsync*, the first viable web-based incremental synchronization solution for cloud storage services. To alleviate the inefficiency of JavaScript execution in web browsers, they also came up with WebR2sync to "flip upside down" the process by offloading computationally intensive block search and comparison operations to the server side. Further optimizations, including location-aware block matching and lightweight checksums, lead to the development of WebR2sync+, which offers an order-of-magnitude performance improvement over WebRsync.

To overcome the "impossible triangle" of efficiency, applicability, and usability in web-based cloud storage services, **Zheng et al.** [72] proposed *WASMrsync+*, a WebAssembly (WASM)-enhanced delta synchronization system that integrates client- and server-side co-optimizations. On the client side, sync-async code decoupling isolates computationally intensive WASM tasks (e.g., MD5 hashing) from asynchronous I/O operations, eliminating unnecessary context-switching overhead. Additionally, streaming compilation accelerates WASM module initialization by leveraging browser streaming APIs to compile code incrementally during download, shortening the entire load time. On the server side, Informed In-place File Construction (IIFC) optimizes memory usage by encoding file updates as dependency graphs, where each operation (e.g., modifying, adding, or deleting) is linked to others based on logical dependencies.

The Dropbox application also uses incremental synchronization in fixed-size blocks. **Li et al.** [50] pointed out that while synchronizing the duplicate content does not incur significant data transfer, such as when 100 MB of identical file content is synchronized, generating only 40 KB of network traffic, frequent small updates can impose session maintenance costs that surpass the actual data transfer overhead, particularly in multi-user file-sharing scenarios. To address this issue, they proposed the *Update-Batch Delayed Sync (UDS)* mechanism, which temporarily buffers all modifications before synchronizing them in batches to the remote server. Furthermore, UDS+ incorporates Linux kernel enhancements to further optimize CPU utilization.

To address the issue of "*abuse of delta sync*" in cloud storage services like Dropbox, **Zhang et al.** [68] proposed *DeltaCFS*, a novel file synchronization framework that adaptively combines delta synchronization with NFS-like file operation interception. By leveraging FUSE (Filesystem in Userspace), DeltaCFS intercepts file operations at the user-space level and dynamically selects optimal sync strategies based on update patterns. For in-place updates (e.g., SQLite writes), DeltaCFS directly captures incremental data through operation interception, eliminating costly file scans and checksum computations. For transactional updates (e.g., Word file saves), which are identified by tracking file operations (e.g., `rename` or `unlink`), it use direct bitwise comparison between old and new versions to replace heavy checksum calculations, thus significantly reducing the computational overhead.

CDC identifies natural chunk boundaries based on the actual data content, such as structural changes or recurring patterns, rather than adhering to a fixed size. This adaptive strategy effectively mitigates the boundary shifting issue commonly observed in FSC method, thereby yielding a higher deduplication ratio. However, CDC is generally more computationally intensive and introduces greater implementation complexity. In the following sections, we review existing synchronization techniques that adopt the CDC paradigm.

To address the challenges of high bandwidth consumption in network file systems, **Muthitacharoen et al.** [49] proposed *LBFS*, a low-bandwidth file system that leverages content-based chunking and similarity detection to minimize data transmission overhead. As shown in Fig. 5, the original file is divided into variable-length blocks by computing a rolling hash over a 48-byte sliding window. A chunk boundary is established whenever the lower 12 bits of the current hash match a predetermined constant. Fig. 5(b) demonstrates that when new content is inserted into a block (e.g., C4), the modification affects only the corresponding chunk, resulting in a new block (e.g., C8). As adjacent blocks remain unchanged, only the newly generated block C8 must be transmitted, thereby minimizing synchronization traffic. In the scenario depicted in Fig. 5(c), the inserted content introduces additional breakpoints, causing the affected region to be split into multiple new blocks (e.g., C9 and C10), both of which need to be sent. Conversely, Fig. 5d illustrates a case where a modification removes existing breakpoints, such as when blocks C2 and C3 are merged into a new block C11, rendering previous segments unusable. Consequently, the entire merged block C11 must be transmitted to maintain file consistency.

To address the high computational overhead and poor adaptability in traditional fixed-size chunking based on Adler32

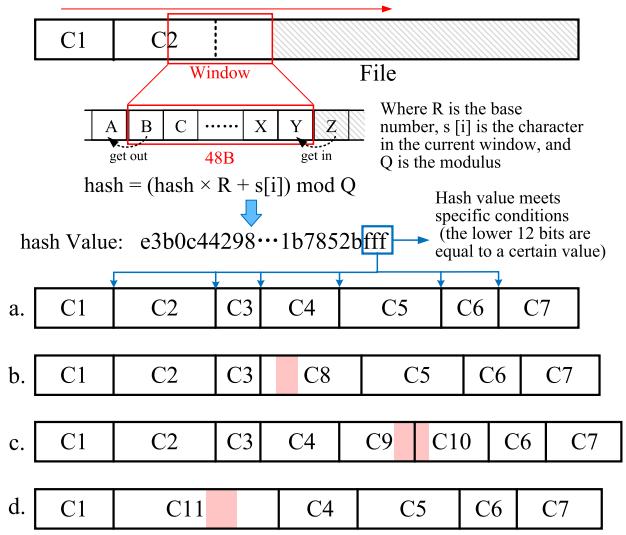


Fig. 5. File chunking and modification handling in LBFS.

hashing, **He et al.** [26] proposed *Dsync* as an optimization to rsync-based synchronization. The core innovation of Dsync lies in replacing Adler32 with lightweight FastFp weak hashes and employing a two-phase verification process—weak hash pre-screening followed by SHA1 strong hash confirmation. Additionally, contiguous chunks that match strong hashes are merged, effectively minimizing computational overhead and reducing redundant metadata transfers.

In mobile cloud backup, the reliance on hash computations for change detection introduces significant computational overhead. To address this issue, **Pan et al.** [75] introduce SolFS, a lightweight, operation-log-based versioning file system that tracks file modifications at the write level. Specifically, SolFS employs per-file mergeable operation logging (MLogging) to record the offset and length of each write, organized using an efficient extent tree. This mechanism accurately captures the locations of modifications within a file, thereby eliminating the need for full-file hash computations.

To address the severe sequential bottlenecks in modern file synchronization, **Zhang et al.** [76] proposed *ParaSync*, a system re-architected for maximum concurrency. First, it decouples boundary identification from checksum computation: multiple threads compute checksums for sub-chunks in parallel, which are then merged into final chunks via a lightweight linear-time algorithm. Second, it introduces a streaming chunk-matching protocol, where the server incrementally streams matching tokens to the client as they are generated, enabling client-side verification process to overlap with server-side computation. Finally, it adopts an absolute-offset-based delta reconstruction mechanism, allowing the server to apply updates in parallel and out of order.

To enhance transfer efficiency for massive spatiotemporal data, **Zhang et al.** [74] proposed *SPsync*, a lightweight synchronization scheme optimized for spatiotemporal data. First, SPsync introduces an odd-even sliding window strategy for CDC, enabling parallel processing by simultaneously sliding over odd and even byte positions, thereby accelerating boundary detection compared to traditional byte-level sliding. To further expedite the chunking process, SPsync integrates with Spark-RDD to cache intermediate hash values generated during sliding-window chunking in memory, reducing disk I/Os meanwhile enabling distributed parallel task execution.

To address synchronization inefficiencies in mobile cloud storage, **Cui et al.** [24] proposed *QuickSync*, a new synchronization system comprising three core components: a network-aware chunker, that dynamically adjusts chunk sizes based on real-time network conditions to balance deduplication efficiency and computational overhead (e.g., 64 KB chunks for low bandwidth vs. 4 MB for high bandwidth); a redundancy eliminator, that employs rolling hash-based feature sketches to identify similar chunks across file versions, thus reducing redundant data transfers via delta encoding; and a batched syncer, that leverages delayed acknowledgments and connection reuse to reduce protocol overhead and maximize throughput.

Building on this, an enhanced synchronization framework called *NetSync* [73] was proposed, incorporating a network adaptive method to optimize data transmission. The system first evaluates real-time network bandwidth via its detection module, which preferentially reuses historical network profiles to minimize measurement overhead, unless significant bandwidth fluctuations are detected. Subsequently, the chunking module adaptively adjusts data block sizes—generating smaller blocks coupled with high-ratio compression algorithms under low-bandwidth scenarios to enhance redundancy elimination, while using larger blocks with lightweight or no compression in high-bandwidth environments to reduce fragmentation costs.

Traditional cloud synchronization methods, such as rsync [48], suffer from high latency, primarily due to redundant multi-round communications, high client-side computational overhead, and server-side concurrency bottlenecks. To address these issues, **Zhang et al.** [70] proposed a new incremental synchronization method, *SimpleSync*, integrating three core innovations: a client-side Cache Data Structure (CDS) that leverages the immutability of backup files in cloud storage to store file checksums locally, eliminating the need for repeated server-client interactions; CDC with Gear fingerprinting, which enhances differential data identification while reducing the computational cost of rolling hash calculations on the client side; Server-side optimization via Kafka buffering for peak shaving and parallel processing with Apache Flink, making the first adaptation of a distributed computing framework for delta synchronization.

"Delta amplification" is a critical challenge for encrypted cloud storage systems that employ delta synchronization. Because each file chunk is encrypted with an independent, content-derived key, even minor data modifications can trigger cascading changes in encryption keys and ciphertext blocks, substantially inflating synchronization traffic. To address this issue, **Wu et al.** [71] proposed *FeatureSync*, a feature-based encryption synchronization scheme that introduces a unified file-level encryption key, as the "feature", to decouple key updates from localized content modifications. This encryption key is typically defined as the maximum hash value within sliding windows. To further mitigate delta amplification effects, FeatureSync also

introduces adaptive window granularity optimization. In specific, fine-grained data chunking is applied near insertion points to contain boundary shift propagation and limit the scope of re-encryption.

As network bandwidth increases, the bottleneck of encrypted cloud synchronization shifts from network latency to computational overhead. In response, **Wu et al.** [27] proposed *FASTSync*, an optimization of FeatureSync aimed at enhancing synchronization efficiency through three key innovations: 1) replacing FeatureSync's Rabin fingerprinting with a lightweight Gear-based hashing algorithm for feature value generation, reducing the computational overhead; 2) replacing the byte-by-byte sliding window comparisons with chunk-level matching based on CDC to accelerate client-server chunk matching process; 3) employing a pipelined three-phase protocol that overlaps client-side chunking, hashing, and encryption with server-side chunk matching and verification, minimizing the idle time through parallelization.

For cloud backup synchronization, standard encryption renders data indistinguishable, whereas deterministic encryption is vulnerable to information leakage. To address this dilemma, **Yang et al.** [77] propose executing the entire data reduction pipeline, including deduplication, delta compression, and local compression, within a hardware-protected enclave (i.e., Intel SGX), where data is decrypted and processed in plaintext. To further improve compression efficiency, the authors introduce a bi-directional delta compression technique. When base chunks exhibit strong locality, it performs forward delta compression, i.e., loading containers in bulk to compress new chunks against existing base chunks. Conversely, it switches to backward delta compression, compressing the scattered old chunks against the new chunks.

2) Full Synchronization: It ensures complete data consistency by transferring the entire data files during each synchronization cycle, thus eliminating the computational overhead associated with incremental synchronization, albeit at the cost of redundant data transmission, as shown in the right portion of Fig. 4. Prominent cloud storage services, such as Google Drive [91] and Microsoft OneDrive [92], rely on full synchronization for maintaining data consistency. However, given the relatively limited body of research exclusively dedicated to full synchronization, this section also incorporates studies that explores hybrid synchronization strategies combining both full and incremental methods.

Existing synchronization approaches exhibit inherent limitations: full synchronization is bandwidth-intensive for large files, while delta synchronization incurs substantial deduplication overhead, particularly for small files. To address this paradox, **Wu et al.** [25] introduced *PandaSync*, a dynamic synchronization framework that adaptively classifies files as "small" or "large" based on network conditions. The classification threshold is determined by the network RRT (R) using the formula: $F = \frac{332}{0.141+0.029\times R}$. For small files, PandaSync employs an optimized full sync that merges sync requests with data transfers to reduce network round-trips. For large files, it switches to delta sync to reduce redundant data transmission. Such a network-aware, adaptive threshold model balances computational and bandwidth costs by aligning synchronization mode transitions with performance inflection points.

To address the inefficiencies of traditional sync methods under fluctuating conditions, **Zhou et al.** [60] proposes *LearnedSync*, a machine learning-driven synchronization framework that dynamically selects sync strategies (FULL/FSC/CDC) by adapting to workload characteristics (e.g., file size, redundancy rate) and environmental parameters (e.g., bandwidth, RTT). The framework integrates three components: a state monitor that collects real-time environmental metrics, a multi-layer perceptron (MLP)-based decision model that identifies the optimal sync method, and a synchronization recorder that validates decisions using ground-truth benchmarks, allowing for iterative model accuracy refinement.

In response to growing demands for data privacy, flexibility, and vendor independence, **Chen et al.** [78] present an open-source synchronization framework that leverages standard cloud object storage APIs to eliminate vendor lock-in while ensuring user-controlled data sovereignty. At its core, the system employs a state-encoding mechanism using quad-tree structures to track file metadata (e.g., filenames, timestamps) across local and cloud environments. By maintaining separate historical and current state trees, the system precisely detects file additions, deletions, and modifications. Synchronization operates through a push-pull paradigm: the "Push" phase identifies local changes by comparing historical and current local trees, uploading new or modified files while pruning obsolete cloud entries; the "Pull" phase mirrors cloud updates locally through hash-based content replication, reducing redundant data transfers by reusing existing files with matched hashes.

C. Discussion

Transmission with lossless compression exhibits a clear evolution—from "general-purpose compression adoption" to "domain-specific optimization", and ultimately to " contextual adaptation"—as illustrated in the upper part of Fig. 7. This progression can be broadly characterized by three major stages:

1) Early systems, such as FogGIS [61], directly apply mature general compression tools (e.g.,. tar.gz), demonstrating the basic feasibility of performing compression at the edge. Similarly, Idrees et al. [64] improve compression ratios by clustering similar data and subsequently applying Huffman coding to each cluster. While these efforts have explored practical ways of using compression, they do not address the underlying design, adaptation, or optimization of the compression algorithms themselves.

2) Specialized compression algorithms has since emerged, represented by Sprintz [62] and MLOC [63]. Sprintz introduces optimizations specifically targeting memory efficiency; however, its applicability remains largely confined to time-series data. MLOC integrates differential-encoding concepts from synchronization into streaming compression, yet its performance depends heavily on semantic redundancy across data versions.

3) More recent systems, such as IoTZip [65] and context-aware compression frameworks [66], [67], push the field toward a new dimension by recognizing that compression is not an isolated algorithmic primitive but a dynamic, context-sensitive

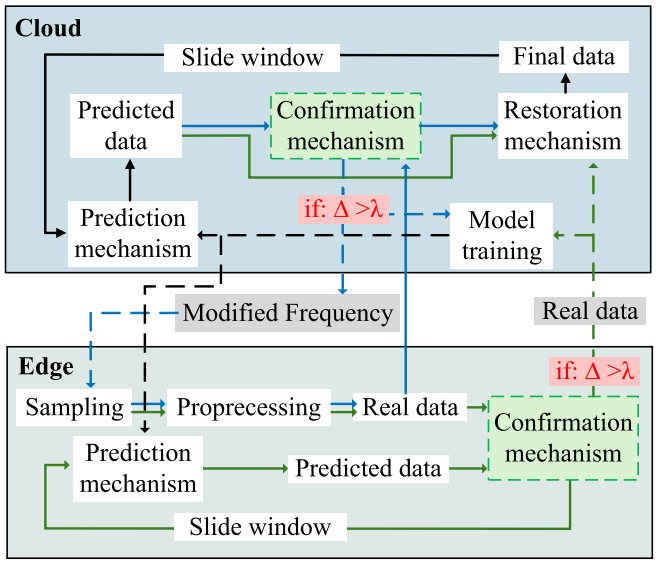


Fig. 6. A cloud-edge collaborative architecture for prediction-driven data transmission.The blue and green arrows in the figure represent two different prediction model strategies (single model and dual model), and the black area represents the overlapping part of the two methods. The solid line represents the "actual data flow", and the dashed line represents the "control instruction flow".

process shaped by network conditions, user behavior, and data characteristics. This shift from an algorithm-centric to a system-centric perspective represents a key step toward achieving adaptive edge intelligence.

The evolution of synchronization systems follows a similar trajectory—from overhead-reducing optimizations to increasingly context-aware adaptation—as also illustrated in the upper part of Fig. 7.

1) Incremental synchronization techniques offer higher efficiency by transmitting only modified data chunks. However, this advantage often comes with increased computational overhead, primarily due to chunking, hashing, and matching. Following the foundational designs of Rsync [48] and LBFS [49], numerous subsequent efforts have sought to reduce this overhead. DSync [26] and FASTSync [27] incorporate FastCDC and Gear Hash to accelerate chunking and hashing, whereas SPsync [74] speeds up both operations using an odd-even parallel sliding-window strategy. However, SPsync's evaluation is based on simulated clusters, leaving its real-world applicability uncertain. DeltaCFS [68] eliminates unnecessary checksum computations through operation-aware optimization; however, migrating its implementation from user space to the kernel may yield further gains. WebR2sync+ [69] shifts computational workload from the client to the server, but this structural change incurs considerable integration complexity. WASM-delta [72] instead accelerates computation directly at the edge using WebAssembly without requiring system changes, though the loading and compilation overhead of WebAssembly remains non-trivial. SolFS [75] alleviates CPU bottlenecks by replacing hashing with a log-structured mechanism; however, it is highly intrusive and difficult to integrate into traditional file systems. ParaSync [76] exploits fine-grained parallelism across chunking, matching, and reconstruction phases to achieve significant speedups, however its dependency on absolute offsets may complicate the handling of frequent file updates. When synchronization interacts with encrypted storage, the issue of "delta amplification" arises. FeatureSync [71] and FASTSync [27] mitigate this by deriving encryption keys from file characteristics (e.g., maximum fingerprint). However, because these features highly depend on file's content, even minor modifications can destabilize derived keys and degrade performance. ShieldReduce [77] achieves high compression efficiency with strong security guarantees using SGX; however, it is tightly coupled with this specific hardware platform, limiting its portability.

2) Beyond reducing per-operation overhead, two additional directions have gained prominence. First, several systems reduce communication overhead by minimizing synchronization rounds. UDS [50] batches small updates before triggering synchronization, while SimpleSync [70] exploits the immutability of cloud-stored files to allow clients to directly generate deltas, thereby reducing round trips. Second, context-awareness has become an increasingly powerful design principle. QuickSync [24] introduce the first network-aware chunking strategy, and NetSync [73] extends this idea by reusing historical network profiles to avoid repeated measurements. However, these systems overlook other important contextual factors—most notably file size. PandaSync [25] incorporates file size to dynamically choose between full and incremental synchronization, showing that under high-bandwidth conditions, full synchronization are superior for small files, where the computation cost outweighs transmission savings. LearnedSync [60] further advances context-aware design by proposing adaptive strategies, including network-aware block sizing and synchronization mode switching. Nonetheless, both approaches neglect the data redundancy: even large, low-redundancy files are poorly suited for synchronization, making redundancy-aware adaptation a promising direction.

V. Lossy Transmission

Lossy transmission enhances data transfer efficiency by selectively discarding less important or low-priority information. This approach is widely adopted across various domains, including health monitoring [21], intelligent transportation [22], [51], industrial domain [52], [53], [54]. The fundamental principle underlying both lossless and lossy transmission is similar, the key distinction lies in the treatment of data fidelity—lossy transmission achieves significantly higher compression rates at the expense of data accuracy. In this section, we examine lossy transmission strategies from two perspectives: 1) transmission based on lossy compression, and 2) transmission guided by predictive modeling techniques.

A. Data Transmission Based on Lossy Compression

The core idea is to preserve the most important data while discarding less important data; however, the definition of "importance" is highly application-dependent.

In IoT systems, data reduction techniques like Perceptually Important Points (PIP) algorithms are effective for compressing static datasets; however, their reliance on a posteriori processing limits their applicability in real-time streaming contexts. To

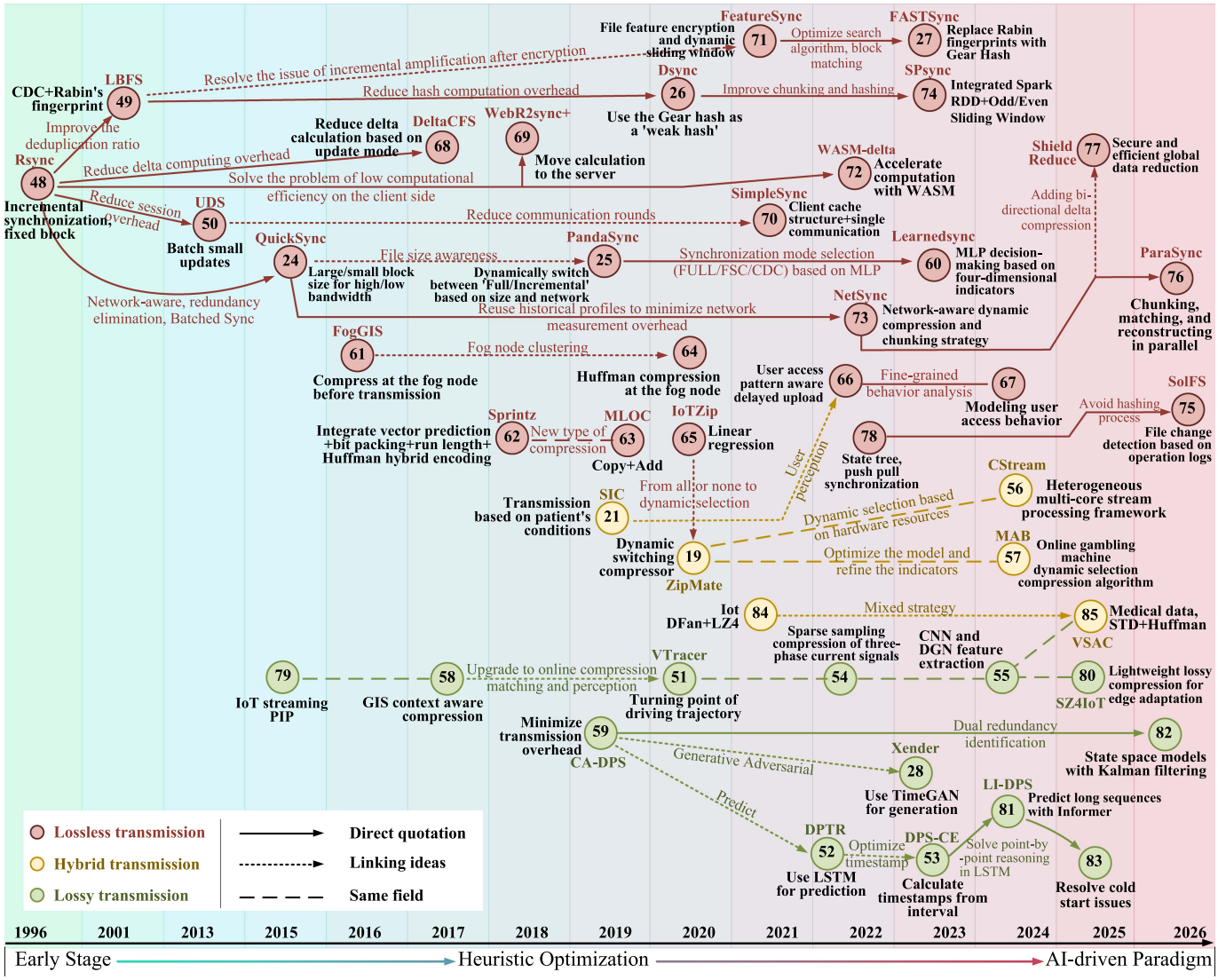


Fig. 7. An illustration of the technical trend in edge-to-cloud transmission domain over the past three decades (1996-2026), highlighting the evolution from early-stage approaches to heuristic optimization, and ultimately to AI-driven paradigms. Each circle, annotated with the corresponding reference number, represents a study reviewed in this article and is positioned according to its publication year, with different colors indicating distinct categories. Labels near the circles in black font describe the technical contributions of each study, while colored text along the connecting lines indicates potential connections between successive studies. The gradually changing background color, from blue to red, visually reflects the growing activity and interest in this field, as evidenced by the rising number of publications.

address this limitation, **Papageorgiou et al.** [79] proposed the *NECtar* agent, an edge computing framework that "streamifies" PIP algorithms through three novel cache projection strategies: CLONE, which replicates the most recent data point; TWIN, which mirrors historical data patterns; and AVG, which forecasts future values via moving averages. These strategies enable on-the-fly importance assessment by extrapolating future trends from cached observations, dynamically identifying critical features (e.g., peaks, troughs) as each data item arrives. By prioritizing the transmission of perceptually important data points, NECTAR effectively reduces data volume without compromising accuracy.

Directly transmitting raw GPS data to data centers presents significant challenges, including high communication latency and bandwidth consumption. To mitigate these issues, **Acharya et al.** [58] proposed a GIS context-aware compression framework for IoT-based Intelligent Transportation Systems (ITS). Their approach dynamically maps vehicle coordinates onto road segments delineated by adjacent reference nodes (e.g., left/right bounds in OpenStreetMap), and encodes positional updates as single-bit indices. Specifically, a "0" index denotes positional continuity within the current road segment, while a "1" signals a boundary crossing. This efficient encoding achieves a 32:1 compression ratio, significantly reducing transmission overhead while maintaining spatial accuracy.

Similarly, **Chen et al.** [51] introduced *VTracer*, an edge-computing-enabled framework that offloads resource-intensive online trajectory compression to drivers' smartphones, mitigating the computational constraints of vehicle-mounted GPS devices. For compression, VTracer retains only critical turning

points at intersections while discarding redundant straight-path segments, thereby achieving near-spatial-lossless representation. Compared to PRESS [93] and CCF [94], under comparable compression ratios, VTracer significantly reduces computational complexity and delivers notable improvements in both time and energy efficiency.

In photovoltaic inverter fault diagnosis scenarios, transmitting massive raw data to the cloud for model training incurs significant bandwidth and latency overhead. To address this issue, **Wang et al.** [54] introduced a compressed sensing-enhanced framework to improve data transmission efficiency. In this approach, edge nodes first utilize Gaussian random matrices to compress raw signals, achieving an 85% reduction in data volume while preserving essential fault features. These compressed samples are then transmitted to the cloud, where a six-layer CNN model with batch normalization is trained, reaching 99.18% accuracy. Once trained, the model is deployed back to edge devices (e.g., NVIDIA Jetson), enabling real-time fault diagnosis with high efficiency.

In image data processing, **Mobarak et al.** [55] proposed the Split-Compression framework, which optimally distributes compression tasks between IoT devices and cloud servers. The framework employs a lightweight encoder on IoT devices, utilizing convolutional layers and Generalized Divisive Normalization (GDN) for efficient data compression, while cloud-based decoders leverage transposed convolutions and inverse GDN layers for high-fidelity reconstruction.

To reduce the high overhead of lossy compression on resource-constrained edge devices, **Idrees et al.** [80] propose *SZ4IoT*, a lightweight adaptation of the SZ algorithm tailored for edge microcontrollers, including ESP32, Teensy 4.0, and RP2040. By removing non-essential components, optimizing memory buffers for 32-bit architectures, and incorporating an interpolation-based prediction mechanism via adaptive curve fitting, SZ4IoT supports efficient local compression of multiple data types (e.g., INT8, INT16, INT32, float, double). This hardware-aware design significantly reduces transmission volume and energy consumption, while maintaining data distortion within user-defined error bounds.

B. Data Transmission Based on Prediction

In contrast, predictive transmission attempts to transmit the model rather than the raw data. Its basic assumption is the inherent patterns of data sequence can be captured by the model.

Data transmission guided by predictive methods typically adopts a cloud-edge collaborative architecture. As illustrated in Fig. 6, two primary prediction frameworks are presented: the green data stream denotes a dual-end model deployment strategy, while the blue data stream corresponds to a cloud-only model deployment approach. 1) In the dual-end deployment paradigm, predictive models are trained on the cloud using data collected from edge devices, employing architectures such as LSTM and TimeGAN to generate synthetic data that closely approximates real-world observations. A lightweight version of the model is deployed at the edge to evaluate prediction accuracy in real time. When the prediction error exceeds a predefined threshold, the actual data is transmitted to the cloud, where the cloud-side model is retrained to enhance prediction accuracy. Updated model parameters are subsequently synchronized back to the edge, ensuring consistency across both ends. 2) In the cloud-only deployment framework, the predictive model is maintained exclusively in the cloud and trained using historical data gathered from edge devices. Both prediction generation and confirmation mechanisms are performed centrally in the cloud. When the prediction error exceeds the specified threshold, the cloud signals the edge device to increase its transmission frequency. This adaptive feedback mechanism enables the model to be refined using more representative input, thereby enhancing overall prediction fidelity.

To alleviate significant bandwidth waste in sensor-to-cloud data transmission, **Sadri et al.** [82] introduce a predictive scheme to eliminate redundant measurements. The core idea is to deploy synchronized state-space models with Kalman filtering at both the fog and cloud layers. It identifies two types of redundancy at the fog layer: temporal redundancy, characterized by negligible changes from previous values, and predictable redundancy, where data points fall within a predefined confidence interval. To maintain model consistency without frequent communication, the system employs a dual-end synchronous update mechanism: redundant measurements trigger model updates using predicted values on both ends; otherwise, the actual data is transmitted for synchronization.

Traditional dual prediction schemes (DPS) reduce communication by transmitting data only when prediction errors exceed a predefined threshold. However, their reliance on static model parameters or stale historical inputs often results in suboptimal performance, particularly under highly dynamic environments. To address this limitation, **Håkansson et al.** [59] proposed a Cost-Aware Dual Prediction Scheme (CA-DPS) that dynamically selects transmission strategies (e.g., transmitting raw data or updating model parameters) based on predicted future transmission costs derived from bootstrapped simulations. A salient feature of CA-DPS is its non-reliance on rigid statistical assumptions. Instead, it synthesizes realistic future data patterns by resampling past prediction residuals in a way that preserves key statistical features of the original data, including average error size, variability, and temporal structure. Both simulation results and real-world data show that CA-DPS outperforms conventional methods (e.g., Constant Model (CM), least-mean-square(LMS), and adaptive method for data reduction(AM-DR) [95]) and significantly reduces the amount of data transmitted.

Deep learning techniques have shown strong capability in handling data with complex patterns. Building on this, **Wu et al.** [52] proposed Deep Learning-based Data Prediction and Transmission Reduction (DPTR), a cloud-edge collaborative framework designed to mitigate the excessive data transmission burden in industrial IoT environments. DPTR leverages synchronized LSTM models on both edge and cloud to predict time-series sensor data. Edge gateways transmit data to the cloud only when prediction errors exceed a predefined threshold. This approach effectively reduces redundant transmissions while preserving data fidelity.

Similarly, **Wu et al.** [53] further extended the predictive paradigm through a dual LSTM-based architecture, wherein models at both the edge and cloud collaboratively optimize

data transmission. A notable enhancement lies in the optimization of temporal information: instead of transmitting full timestamp sequences, only the initial two timestamps are sent, with subsequent timestamps inferred computationally. Sensor readings are selectively transmitted only when prediction errors surpass a dynamically adjusted threshold. Experimental results demonstrate that this method effectively alleviates bandwidth constraints in smart factories while maintaining the accuracy required for higher-level applications.

In Industrial Internet of Things (IIoT) scenraios, traditional DPS often struggle with excessive model inference overhead and diminished forecasting accuracy over long sequences. To address these limitations, **Wu et al.** [81] proposed a novel long-sequence dual prediction scheme, *LI-DPS*, built upon the Informer model [96]. The LI-DPS framework adopts a cloud-edge collaborative architecture to balance computational load and transmission efficiency. Specifically, the cloud is responsible for training a complex Informer-based prediction model using historical data and subsequently synchronizes a lightweight version to the edge, thereby ensuring consistency between both ends. At the edge, the simplified model is employed for real-time inference, producing multiple future values per inference cycle. This design significantly reduces inference frequency and minimizes data transmissions by reporting only those data points whose prediction errors surpass a predefined threshold.

In the predictive data reduction paradigm, data transmission is triggered only when predictions deviate from actual readings beyond a predefined tolerance threshold. However, in remote scenarios such as smart agriculture, newly deployed edge nodes often suffer from a "cold-start problem" due to the lack of sufficient historical data for model training. To address this issue, **Kreković et al.** [83] propose pretraining the LSTM model on large-scale publicly available satellite-derived climate data (i.e., opernicus ERA5-Land), enabling effective cross-site generalization and deployment in new locations without requiring local data collection or retraining.

The *Xender* system, proposed by **Liu et al.** [28], represents another innovative prediction method. On IoT devices, TDSam dynamically samples critical points (e.g., peaks, troughs, abrupt changes) using a threshold-based algorithm, minimizing data transmission while preserving temporal dynamics. On the cloud, TDGen, an enhanced variant of TimeGAN, Leverages dilated causal convolution, attention-based decoding, and bidirectional RNNs to reconstruct high-fidelity data. Xender's innovation lies in its dual adaptation mechanism: 1) Content-aware adaptation mechanism that periodically assesses data similarity via Dynamic Time Warping (DTW), triggering model updates and adjusting the sampling rate when data quality deteriorates; 2) Computation-aware anytime generation enables quality-tiered outputs (Low/Medium/High) through layer bypassing.

C. Discussion

Generally, both lossy compression and predictive transmission can achieve higher efficiency than lossless methods by introducing application-tolerable information loss. While sharing this conceptual foundation, the two represent fundamentally different design philosophies: compression-based methods selectively preserve semantically critical information while discarding redundant content, whereas predictive methods aim to infer future data from historical patterns. Collectively, they drive the evolution from "heuristic-based compression" toward "learning-driven adaptation", as illustrated in the bottom part of Fig. 7.

In lossy compression, although the guiding principle is intuitive—retain what is important—the operational definition of "importance" is highly application-dependent. In IoT sensing systems, for example, NECtar [79] identifies salient peaks and troughs through on-the-fly importance assessment. However, its performance remains sensitive to prediction accuracy. In trajectory compression, schemes such as GIS-Aware Compression [58] and VTracer [51] preserve only key turning points while eliminating redundant straight segments; however, their efficacy depends heavily on map quality and often deteriorates in complex intersections. In industrial fault diagnosis, CS-CNN [54] achieves high compression ratios on three-phase current signals by only preserving diagnostically relevant features. For image-processing workloads, Split-Compression [55] prioritizes learned feature representations that support high-fidelity reconstruction. Both of these studies further suggest that lightweight model designs are needed for practical deployment on constrained edge devices. Idrees [80] achieves ultra-lightweight lossy compression on microcontrollers; however, its absolute error truncation removes critical high-frequency features, thereby compromising the efficacy of downstream tasks such as anomaly detection.

Predictive transmission is grounded in a different assumption: the inherent patterns in data sequences can be faithfully captured by suitable models. Representative systems highlight both progress and limitations. CADPS [59] dynamically selects between transmitting raw data and updating model parameters based on predicted future transmission costs, yet its simple model struggles with complex temporal patterns. To improve prediction quality, Wu et al. [52] employ LSTM networks to reduce transmission volume while controlling relative error, though the scheme does not optimize timestamp transmission. Subsequent work [53] explicitly addresses timestamp-related overhead but assumes fixed intervals, limiting its effectiveness under dynamic sampling. Wu et al. [81] adopt the Informer architecture to address the high inference latency of LSTMs on long sequences; nonetheless, the model remains relatively large for resource-constrained devices. Xender [28] introduces a promising alternative: the edge transmits only sparse keypoint samples, while the cloud reconstructs the full time series using TimeGAN. While promising, current support is limited to univariate signal generation; future extensions may incorporate multivariate generation and cross-variable correlations. Furthermore, Sadri [82] significantly reduces transmission frequency via dual-ended prediction, but its strict edge-cloud synchronization makes it highly vulnerable in highly dynamic or asymmetric edge environments. Kreković [83] addresses the "cold-start" problem via satellite-data pretraining, yet its static error threshold may limit adaptability in highly dynamic multi-sensor agricultural environments.

VI. Transmission Integrating Lossy and Lossless Methods

In this section, we focus on hybrid schemes. Unlike purely lossy methods that simply discard low-priority data, hybrid approaches adopt a more nuanced treatment of data fidelity, ensuring that high-priority data is transmitted with high precision, while lower-priority data is maintained at reduced precision.

A. Data Transmission Based on Hybrid Lossy and Lossless Compression

The following studies generally formulate edge-to-cloud transmission as a multi-objective optimization problem.

To address the transmission bottleneck in sensor-cloud systems, **Lu et al.** [84] proposed a hybrid compression method, *DFan*, that integrates the benefits of both lossy and lossless compression. At its core, the DFan algorithm enhances conventional approaches by dynamically analyzing sequential data samples. Rather than indiscriminately discarding linearly trending points, DFan preserves critical inflection points (e.g., abrupt spikes) when deviations exceed predefined thresholds, while approximating linear trends within acceptable error margins. Numerical data undergoes keypoint-preserving lossy compression followed by LZ4-based lossless compression of residual errors, whereas non-numerical data is directly compressed using LZ4. Although alternative techniques, such as wavelet transform and Discrete Cosine Transform, may achieve higher lossy compression ratios, they typically incur greater reconstruction errors.

In health monitoring scenarios, existing systems lack adaptive compression to balance real-time processing requirements and efficient data reduction. To address this challenge, **Andrade et al.** [85] proposed the Vital Sign Adaptive Compressor (VSAC) model, combining lossy and lossless compression within a hierarchical architecture. At the edge layer, the Swinging Door Trending (SDT) algorithm performs lossy compression by dynamically filtering redundant vital sign data points (e.g., stable heart rate readings) based on configurable deviation thresholds. Upon detecting critical anomalies, such as sudden arrhythmias, the system dynamically tightens thresholds to ensure diagnostic fidelity. At the fog layer, a size-aware lossless compression strategy is applied: Huffman coding optimizes small files (e.g., $<$33.2 KB) for rapid processing, whereas LZW is employed for larger files to maximize compression efficiency.

To address the high energy consumption and latency in smart healthcare data transmission, **Abdellatif et al.** [21] developed an edge-based epilepsy monitoring system that processes EEG signals from wearable devices through a three-tier architecture (IoT-edge-cloud). The system integrates lightweight feature extraction and a fuzzy logic-based classification algorithm (SIC) to distinguish seizure states with high efficiency. For data transmission, it employs an adaptive strategy optimized for varying neurological conditions: during active seizures (AC), raw EEG data is transmitted to ensure diagnostic precision; in non-active chronic states (NAC), a threshold compression method based on discrete wavelet transform (DWT) reduces data volume while preserving essential information; and during seizure-free (SF) periods, ultra-compact feature vectors are utilized to minimize transmission overhead. This dynamic compression demonstrates a latency-energy-accuracy balanced solution for real-time epilepsy monitoring.

In terms of system-level optimization of hardware awareness, **Zeng et al.** [56] proposed *CStream*, a framework for parallel stream compression on multi-core edge devices leveraging software-hardware co-design. CStream dynamically selects compression algorithms based on application needs: lossless methods (e.g., LEB128-NUQ, Tcomp32) preserve high fidelity for critical data (e.g., ECG monitoring data), while lossy approaches (e.g., ADPCM, PLA) achieve higher compression ratios with minimal information loss (e.g., Rovio game interaction data). For hardware adaptation, CStream employs asymmetry-aware scheduling on heterogeneous processors (e.g., RK3399), where compression tasks are decomposed into fine-grained operations and assigned to cores based on their computational capabilities, while balanced allocation is used for symmetric multi-core processors.

Existing IoT data transmission methods often target specific scenarios, lacking a unified framework that balances compression efficiency, data heterogeneity, and dynamic resource constraints. To address this limitation, **Lu et al.** [19] propose *ZipMate*, an adaptive compression middleware designed to optimize data compression on edge devices. Their study reveals significant performance variations among compressors (e.g., gzip, ZSTD, SZ, WebP) across datasets, highlighting the necessity of an dynamic selection strategy. ZipMate adapts to two optimization scenarios: computation offloading, which minimizes transfer time, and storage offloading, which maximizes the compression ratio. It models compression throughput under CPU, storage I/O, and network bandwidth constraints, selecting the optimal compressor based on the dataset characteristics and resource availability. To achieve this, ZipMate employs lightweight sampling techniques (e.g., prefix-based) to predict compression ratios without full-dataset processing.

Liu et al. [57] proposed *AdaEdge*, a dynamic compression selection framework designed for resource-constrained IoT environments. AdaEdge addresses the inefficiency of static "one-size-fits-all" compression techniques by dynamically adapting to varying data workloads and hardware constraints. The framework employs a multi-armed bandit (MAB) algorithm to select optimal compression strategies, both lossless (e.g., Gorilla, Sprintz) and lossy (e.g., PAA, FFT), based on system feedback such as compression ratio, throughput, and task accuracy. AdaEdge operates in two modes: online mode, which prioritizes lossless compression when bandwidth is constrained and switches to lossy methods when necessary, and offline mode, which aggressively compresses older data using LRU-based policies to maintain storage limits. The framework supports flexible optimization objectives, including query accuracy, machine learning performance, or hybrid goals.

B. Discussion

Early systems such as [21], [85] adjust transmission strategies based on patient status, yet still rely on manually tuned parameters and thus lack true self-adaptivity. ZipMate [19]

dynamically selects compression algorithms according to runtime conditions (e.g., network bandwidth, disk I/O throughput, compression ratio), but its performance prediction depends on a sampling mechanism that can substantially degrade accuracy. CStream [56] further advances this direction by decomposing compression pipelines into fine-grained operations and distributes them across heterogeneous processing cores in proportion to their computational capabilities. However, this approach has been demonstrated primarily on the six-core heterogeneous RK3399 processor and may be less efficient on homogeneous or differently configured processors. To address task diversity, AdaEdge [57] employs a MAB framework to select the optimal compression algorithm under multiple objectives; however, its decision space remains restricted to algorithm selection, leaving additional dimensions—such as data modality, resource constraints—underexplored. DFan [84] integrates lossy compression with lossless residual compression, maintaining high compression ratios while bounding reconstruction error, though its performance deteriorates on highly nonstationary signals. Building on this idea, prior study [85] performs lossy compression to extract critical information and subsequently applies Huffman or LZW coding to balance computational overhead and compression efficiency. Nonetheless, more advanced entropy-coding techniques could further improve performance.

In summary, edge-to-cloud transmission systems are expected to evolve into intelligent agents capable of perceiving environmental context, assessing the information value of data, and dynamically orchestrating hybrid strategies. This represents a shift in system objectives—from merely optimizing bit-level transmission efficiency to effectively preserving and maximizing the value of information.

VII. A Unified Optimization Framework

Edge-to-cloud data transmission fundamentally constitutes a multi-objective trade-off problem. In this section, we use bandwidth usage B, energy consumption E, and end-to-end latency L as representative system-level objectives. Other system resources, such as CPU and memory, are treated as explicit constraints. Meanwhile, data quality Q must satisfy application-specific requirements, and its effective value is modeled by a non-decreasing utility function $U(Q)$.

A. Cost-Centric Optimization Framework

To unify the heterogeneous objectives, we propose a cost-centric optimization framework in which both resource usage and performance degradation are treated as components of a generalized system cost. In this view, latency is interpreted as a performance-oriented penalty associated with delayed data delivery. Specifically, bandwidth is chosen as the reference unit, while time-varying coefficients $\beta(t)$ and $\gamma(t)$ serve as implicit conversion factors that map energy and latency into a bandwidth-equivalent cost space. Formulating these metrics in a utility-normalized cost framework allows us to capture unit-utility efficiency. The resulting optimization problem for the transmission strategy $\theta \in \Theta$ over the time horizon $[0, T]$ is:

$$\min_{\theta} C_{\text{weighted}}(\theta) = \int_0^T \left(\frac{B(t;\theta) + \beta(t) E(t;\theta)}{U(Q(t;\theta))} + \frac{\gamma(t) L(t;\theta)}{U(Q(t;\theta))} \right) w(t)\, dt, \quad (1)$$

subject to:

$$Q(t;\theta) \geq Q_{\min}(t), \quad \forall t \in [0,T] \quad (2)$$

$$C_{\text{CPU}}(t;\theta) \leq C_{\text{CPU}}^{\max}, \quad C_{\text{MEM}}(t;\theta) \leq C_{\text{MEM}}^{\max}, \quad \forall t \in [0,T] \quad (3)$$

$$\int_0^T E(t;\theta)\, dt \leq E_{\text{total}}^{\text{budget}} \quad (4)$$

$$L(t;\theta) \leq L_{\max}, \quad \forall t \in [0,T] \quad (5)$$

$$B(t;\theta) \leq B_{\max}(t), \quad \forall t \in [0,T] \quad (6)$$

To ensure both interpretability and practical deployability, we further specify model parameters as follows.

System-Perceived Metrics: $B(t;\theta)$, $E(t;\theta)$, and $L(t;\theta)$ denote bandwidth usage, energy consumption, and end-to-end latency, respectively. These metrics can be obtained through lightweight system monitoring and application-level instrumentation. For example, bandwidth is estimated from network interface byte counters (e.g., `/proc/net/dev`) using time-windowed differences; latency is measured via application-level timestamps or round-trip time (RTT) probing (e.g., `ping`, request-response timing); CPU and memory usage are obtained from standard operating system interfaces (e.g., `/proc/stat`, `/proc/meminfo`); energy consumption is collected using external measurement tools, such as the Monsoon Power Monitor [97].

Application-Defined Metrics: $Q(t;\theta)$ denotes the application-level data quality induced by the transmission strategy θ, capturing the fidelity and task relevance of transmitted data for downstream applications. This abstract, application-dependent metric can be instantiated via domain-specific measures such as reconstruction fidelity, semantic consistency, or feature preservation. $U(Q(t;\theta))$ is a non-decreasing utility function that maps data quality to task value, such as inference accuracy or task-specific effectiveness, ensuring that higher-quality data yields no lower utility. It serves as a task-level reward function and can be instantiated in different forms depending on application requirements. The time-varying weight $w(t)$ is a non-negative function that models temporal variations in data importance, such as workload bursts or event-driven scenarios.

Derived Parameters: The weighting coefficients $\beta(t)$ and $\gamma(t)$ quantify the relative cost of energy and latency with respect to bandwidth. Two complementary strategies are adopted to instantiate these coefficients:

1) Static normalization-based weighting assigns relative weights based on long-term system metrics, capturing global resource scarcity and performance constraints in predictable environments. The weights are derived from normalized aggregate

metrics over the planning horizon:

$$\beta = \frac{E_{\text{total}}/E_{\max}}{B_{\text{total}}/B_{\max}}, \quad \gamma = \frac{L_{\text{total}}/L_{\max}}{B_{\text{total}}/B_{\max}}. \tag{7}$$

Here, $\beta > 1$ indicates that energy is relatively more scarce than bandwidth, which leads to a higher cost penalty on energy consumption and thus discourages excessive use. Similarly, γ scales the latency penalty relative to bandwidth. This "Global Budget Normalization" ensures that heterogeneous quantities with different units (e.g., Joules, Seconds) are mapped into a unified cost framework. The approach is well-suited for scenarios with predictable workloads and stable operating conditions, such as simple compression-based systems (e.g., FogGIS [61]) and predictive methods (e.g., CADPS [59], Xender [28]).

2) Dynamic elasticity-based weighting captures the marginal impact of bandwidth, energy, and latency on overall utility in real time, making it suitable for dynamic environments.

$$\epsilon_{U,R}(t) = \frac{R}{U(Q)} \cdot \frac{\partial U(Q)}{\partial R}, \tag{8}$$

Borrowed from economics, this represents the "Marginal Utility", quantifying the responsiveness of utility $U(Q)$ to changes in system metric R (e.g., B, E, and L). The weighting coefficients are computed as:

$$\beta(t) = \frac{\epsilon_{U,E}(t)}{\epsilon_{U,B}(t)}, \quad \gamma(t) = \frac{\epsilon_{U,L}(t)}{\epsilon_{U,B}(t)}. \tag{9}$$

This formulation prioritizes quantities with higher marginal utility gain per unit cost. For example, $\beta(t) > 1$ indicates that, at time t, allocating additional energy is more effective than consuming an equivalent amount of bandwidth. Such elasticity-aware weighting is well suited to context-aware systems [21], [85] and adaptive transmission frameworks [28], [57].

B. Scenario-Specific Method Selection

Beyond the two representative operational regimes discussed above, transmission methods can also be selected more directly for specific scenarios. Specifically, Table I can be used to identify candidate methods that match data fidelity requirements and align with data types and hardware platforms, while Table II further refines this selection according to the targeted optimization objectives. For instance, trajectory data are typically generated and transmitted by mobile devices, where efficient compression is critical and certain degree of data loss is acceptable. Under such conditions, VTracer [51] can effectively reduce transmission volume while preserving essential semantic information by retaining only key turning points.

In addition, selecting an optimal transmission strategy often requires a deeper understanding of contextual factors and inherent trade-offs, as discussed in Sections IV-C, V-C, and VI-B. For example, under stable, high-bandwidth conditions, full synchronization may outperform incremental methods for small files due to its lower computational overhead [25]. In contrast, highly dynamic networks benefit more from adaptive approaches, such as QuickSync [24] and LearnedSync [60].

C. Unified Evaluation Metric

To enable comprehensive comparison across diverse transmission strategies, we propose the Edge-Cloud Transmission Score (ECTS). ECTS consolidates multiple dimensions into a single, normalized score, facilitating direct comparison among lossless, lossy, and hybrid approaches. The core formulation is a weighted sum of normalized metrics:

$$\text{ECTS} = \sum_{i=1}^{n} w_i \cdot \mathcal{N}(M_i) \tag{10}$$

where M_i denotes a raw metric (e.g., energy consumption, latency), w_i represents its application-defined weight ($\sum w_i = 1$), which can be adjusted according to different applications, and $\mathcal{N}(\cdot)$ a normalization function mapping all metrics to a comparable scale $[0, 1]$.

Normalization is defined separately for *benefit* (larger-is-better) and *cost* (smaller-is-better) metrics:

$$\mathcal{N}_{\text{benefit}}(M_i) = \frac{M_i - M_i^{\min}}{M_i^{\max} - M_i^{\min}} \tag{11}$$

$$\mathcal{N}_{\text{cost}}(M_i) = \frac{M_i^{\max} - M_i}{M_i^{\max} - M_i^{\min}} \tag{12}$$

$M_i^{\max}$ and $M_i^{\min}$ are the maximum and minimum observed values for metric i across all strategies within an experiment.

For lossless transmission, ECTS typically includes metrics such as compression ratio, throughput, latency, and resource usage. For lossy or hybrid strategies, a data fidelity metric (e.g., Euclidean distance, DTW) is incorporated as a critical benefit metric, ensuring that efficiency gains do not compromise application-level utility.

VIII. Limitations of Existing Studies

Despite recent advances in existing approaches, several inherent limitations persist, hindering their practical effectiveness.

- *Fragmented awareness and insufficient multi-objective trade-offs:* This represents a fundamental limitation in existing research. Edge-to-cloud transmission is inherently a resource-constrained, multi-objective optimization problem. System resources, including bandwidth, computation, memory, and energy, are dynamic and tightly coupled; optimizing a single objective (e.g., data reduction) in isolation often leads to bottleneck shifting (e.g., from bandwidth to CPU). Moreover, the relative importance of optimization objectives varies significantly across scenarios; for example, latency-sensitive applications prioritize rapid processing, whereas energy-constrained edge sensing favors lightweight computation.

However, most existing studies exhibit fragmented awareness, resulting in partially optimized solutions, as evidenced in Table II. Specifically, they fail to explicitly model multi-dimensional resource constraints within a unified framework, limiting their ability to adapt to dynamic fluctuations in edge environments. In addition, they lack task-awareness, often relying on fixed optimization weights that are not aligned with varying application requirements. Consequently, the trade-offs among competing objectives are either implicitly handled or

determined by fixed heuristics, rather than being explicitly and adaptively optimized in a unified manner. For example, the prevailing literature remains heavily bandwidth-centric [28], [49], [50], [60], [64], [71], [85], leaving the critical interplay among resource dimensions and task-specific priorities largely underexplored.

• *Model limitations in AI deployment:* Modern edge-to-cloud transmission increasingly relies on AI techniques, which are primarily used for two purposes: dynamic resource adaptation [57], [60] and approximate data generation [28]. However, current AI-driven approaches face three key challenges: 1) *High training overhead for model adaptation.* Performance-sensitive transmission systems often operate in an online mode to adapt to rapidly changing environments [28], [57], [60], [73]. A fundamental trade-off emerges between reducing retraining overhead and preserving transmission efficiency. 2) *Limited scope of resource adaptation.* Existing methods typically consider only a narrow set of factors, resulting in sub-optimal adaptation. For example, AdaEdge [57] selects among different compression algorithms, whereas LearnedSync [60] adjusts only synchronization modes, even though other parameters, such as chunk size, thread scheduling or batching strategies, also significantly influence system performance. 3) *Constraints of generative models.* Current generative approaches, such as TimeGAN used in Xender, predominantly focus on single-variable generation, while overlooking complex multi-variable dependencies and cross-correlations that are common in real-world workloads.

• *Lack of unified evaluation platforms and datasets:* Approximately 65% of the reviewed studies do not provide open-source implementations, including several inspiring efforts such as Xender [28], LearnedSync [60], and NetSync [73]. Although the remaining 35% of studies release code, the implementations are developed in heterogeneous programming languages, making direct and consistent evaluation challenging. For example, FASTSync [27] is implemented in Go, while ZipMate [19] is written in Python. On the data side, even within comparable application scenarios, studies frequently rely on distinct datasets, further complicating fair comparison. For instance, in data synchronization tasks, FeatureSync [71] evaluates on Linux kernel data, whereas FASTSync [27] adopts open-source software repositories.

• *Limited transferability:* Existing techniques often struggle when applied across different scenarios, owing to three key challenges. First, different data modalities exhibit high heterogeneity in structural semantics and redundancy patterns. For example, data synchronization excels with highly redundant files but perform poorly on time-series data, where the fine-grained variability in floating-point values prevents effective deduplication at large granularities. Second, the edge hardware ecosystem is highly heterogeneous, with over 30 device types represented in the reviewed studies (Table I). Such a wide variation in computing capability, memory capacity, and architectural features makes it difficult to achieve the ideal of "train once, deploy everywhere." Third, the relative importance of optimization objectives varies fundamentally. For example, Sprintz [62] prioritizes memory efficiency, whereas many other compression approaches [61], [63], [64], [65], [66], [67] do not consider memory usage at all.

• *Overprovisioning:* Most existing studies, particularly those concerning lossy transmission [28], [84], [85], aim to preserve high data fidelity using evaluation metrics such as DTW. However, these approaches often overestimate the fidelity truly required by real applications, leading to unnecessary overprovisioning. For instance, many predictive models only need to detect specific temporal patterns rather than reconstruct a perfectly accurate signal [57]. As such, the emphasis on raw bit-level accuracy over information value leaves substantial room for further data reduction, while still fully satisfying downstream requirements.

• *Insufficient collaborations:* Most existing studies overlook the potential benefits of collaborative device networks. Our analysis reveals substantial heterogeneity among edge devices, spanning over 30 device types, which creates new opportunities for dynamic task scheduling under highly dynamic workload distributions. For instance, resource-rich devices could assist lower-capability devices by offloading computationally intensive tasks, while heavily loaded edge nodes may delegate transmission tasks to underutilized peers. However, such collaborative mechanisms remain largely underexplored in existing systems. Moreover, although edge devices benefit from proximity to data sources, cloud layers provide abundant computational resources; nevertheless, effective coordination between edge and cloud remains limited.

IX. Future Outlook

In this section, we outline several promising research directions derived from our analysis of existing limitations, emerging technical trends, and a holistic understanding of this field.

• *Awareness-driven adaptive paradigm:* The aforementioned limitations, including fragmented resource awareness, limited transferability, overprovisioning, and lack of collaborations, stem from a common root cause: the absence of unified, awareness-driven paradigm. Specifically, we identify three key dimensions: 1) *Resource awareness*, which captures the dynamic states of system resources, such as computation, memory, bandwidth, and energy, enabling constraint-aware adaptation to ensure feasibility and efficiency under limited resources. 2) *Task awareness*, which represents a shift from data-centric to semantic-centric design by aligning transmission objectives with application requirements, prioritizing task-relevant information over raw data fidelity, and thereby avoiding unnecessary overprovisioning. 3) *Context awareness*, which captures variations in external workloads and operating environments, enabling adaptive transmission strategies in response to changing environments (e.g., sudden data bursts). These forms of awareness can be further extended beyond individual devices to support collaborative optimization, including adaptive task scheduling across edge devices and dynamic workload partitioning between edge and cloud.

• *Dynamic multi-objective trade-offs:* Building upon the awareness-driven paradigm, edge-to-cloud transmission should explicitly model and resolve dynamic trade-offs among

heterogeneous objectives, including task performance, throughput, latency, compression efficiency, computational overhead, memory consumption, and energy usage. Within this framework, trade-offs are not predefined but instead emerge from the interaction of resource availability, context variability, and task semantics. Consequently, this requires optimization policies capable of responding to fluctuating conditions. Learning-based approaches, such as reinforcement learning, offer a promising direction by capturing complex system dynamics and deriving adaptive policies from feedback signals. In parallel, incorporating explicit cost models to quantify the interplay among computation, transmission, and task performance can provide a structured foundation for the optimization process.

• *Towards more efficient AI-driven transmission:* Addressing the limitations of current AI-driven transmission opens several promising research directions. 1) *Advancing multi-variate generation.* Moving beyond single-variable generation paradigms, future research should emphasize modeling complex multi-dimensional dynamics, including inter-sensor dependencies and cross-correlations, to improve fidelity, consistency, and robustness. 2) *Optimizing training efficiency.* To mitigate the high overhead associated with online model adaptation, it is essential to balance predictive accuracy with adaptation cost, particularly in latency-sensitive applications. This can be achieved through lightweight model architectures, reduced training iterations, and effective utilization of hardware acceleration (e.g., specialized AI accelerators or high-performance GPUs). 3) *Integrating LLMs.* The "zero-shot" or "few-shot" capabilities of LLMs can help to reduce dependency on large-scale labeled data, while their strong contextual reasoning enables more intelligent and context-aware decision-making.

• *Standardization of evaluation platforms and datasets:* To address fragmentation in current research, future efforts should prioritize the development of a standardized evaluation ecosystem. This involves three key directions: 1) developing a unified, open-source benchmarking platform that captures diverse, real-world transmission scenarios for consistent performance comparison; 2) curating standardized datasets across multiple application domains to ensure fair and reproducible evaluations; and 3) fostering a collaborative culture in which researchers commit to releasing both code and data under common frameworks. By aligning on these standards, the community can accelerate innovation, reduce redundant efforts, and establish a shared foundation for robust and generalizable AI-driven transmission systems.

• *Domain-specific versus general-purpose:* Another critical observation arises from the prevalence of time-series data, which accounts for a substantial portion of the reviewed studies (over 40%). Within this subset, several domain-specific compression methods have been proposed, represented by lossless approaches such as Sprintz [62] and lossy techniques such as Xender [28] and LI-DPS [81]. Meanwhile, recent advancements in time-series-tailored lossless compression—e.g., Gorilla [98], Chimp [99], and Elf [100]—have demonstrated substantial improvements in compression efficiency. However, direct and systematic comparisons between them are still lacking. As such, it remains unclear whether modern lossless compressors consistently outperform earlier techniques across all key metrics, including compression ratio, performance, memory efficiency, and energy consumption. If this is the case, the advantage of prior methods, particularly lossy compression schemes, could be considerably diminished. Addressing this gap represents a natural direction for our future work.

X. Conclusion

This paper presents a comprehensive review of recent optimization strategies for edge-to-cloud data transmission. The discussion is organized into three categories: lossless, lossy, and hybrid approaches. For each category, we systematically examine representative studies and provide an in-depth analysis of their respective advantages and limitations. Building on this foundation, we further trace and visualize the technical evolution of the field over the past three decades, helping readers clearly grasp the trends and relationships among successive studies. Finally, we outline several limitations and promising directions for future research.

References

[1] T. Ahmad and D. Zhang, "Using the Internet of Things in smart energy systems and networks," *Sustain. Cities Soc.*, vol. 68, 2021, Art. no. 102783.
[2] M. Shengdong, X. Zhengxian, and T. Yixiang, "Intelligent traffic control system based on cloud computing and Big Data mining," *IEEE Trans. Ind. Informat.*, vol. 15, no. 12, pp. 6583–6592, Dec. 2019.
[3] L. P. Qian, Y. Wu, B. Ji, L. Huang, and D. H. K. Tsang, "HybridIoT: Integration of hierarchical multiple access and computation offloading for IoT-based smart cities," *IEEE Netw.*, vol. 33, no. 2, pp. 6–13, Mar./Apr. 2019.
[4] "IoT signals report: IoT's promise will be unlocked by addressing skills shortage, complexity and security," 2019. [Online]. Available: https://blogs.microsoft.com/blog/2019/07/30
[5] T. Saheb and L. Izadi, "Paradigm of IoT Big Data analytics in the healthcare industry: A review of scientific literature and mapping of research trends," *Telematics Inform.*, vol. 41, pp. 70–85, 2019.
[6] A. Li et al., "CloudCMP: Comparing public cloud providers," in *Proc. 10th ACM SIGCOMM Conf. Internet Meas.*, 2010, pp. 1–14.
[7] I. Drago et al., "Inside dropbox: Understanding personal cloud storage services," in *Proc. 2012 Internet Meas. Conf.*, 2012, pp. 481–494.
[8] I. Drago et al., "Benchmarking personal cloud storage," in *Proc. 2013 Conf. Internet Meas. Conf.*, 2013, pp. 205–212.
[9] B. -Y. Ooi, Z. -W. Kong, W. -K. Lee, S. -Y. Liew, and S. Shirmohammadi, "A collaborative IoT-gateway architecture for reliable and cost effective measurements," *IEEE Instrum. Meas. Mag.*, vol. 22, no. 6, pp. 11–17, Dec. 2019.
[10] Q. Zhu, R. Wang, Q. Chen, Y. Liu, and W. Qin, "IoT gateway: Bridging wireless sensor networks into Internet of Things," in *Proc. IEEE/IFIP Int. Conf. Embedded Ubiquitous Comput.*, 2010, pp. 347–352.
[11] W. T. Chai, B. Y. Ooi, S. Y. Liew, and S. Shirmohammadi, "Taxi-sharing: A wireless IoT-gateway selection scheme for delay-tolerant data," in *Proc. IEEE Int. Instrum. Meas. Technol. Conf.*, 2018, pp. 1–6.
[12] G. S. Fischer, R. da Rosa Righi, V. F. Rodrigues, and C. A. da Costa, "Use of Internet of Things with data prediction on healthcare environments: A survey," *Int. J. E-Health Med. Commun.*, vol. 11, no. 2, pp. 1–19, 2020.
[13] M. Xiang, D. Fu, and K. Lv, "Identifying and predicting trends of disruptive technologies: An empirical study based on text mining and time series forecasting," *Sustainability*, vol. 15, no. 6, 2023, Art. no. 5412.
[14] D. S. K. Karunasingha and S.-Y. Liong, "Enhancement of chaotic hydrological time series prediction with real-time noise reduction using extended Kalman filter," *J. Hydrol.*, vol. 565, pp. 737–746, 2018.
[15] B. Lim, S. Zohren, and S. Roberts, "Recurrent neural filters: Learning independent Bayesian filtering steps for time series prediction," in *Proc. 2020 Int. Joint Conf. Neural Netw.*, 2020, pp. 1–8.
[16] İbrahim Kök and S. Özdemir, "DeepMDP: A novel deep-learning-based missing data prediction protocol for IoT," *IEEE Internet Things J.*, vol. 8, no. 1, pp. 232–243, Jan. 2021.

[17] H. Harb, C. A. Jaoude, and A. Makhoul, "An energy-efficient data prediction and processing approach for the Internet of Things and sensing based applications," *Peer-to-Peer Netw. Appl.*, vol. 13, no. 3, pp. 780–795, 2020.

[18] H. Liazid, M. Lehsaini, and A. Liazid, "Data transmission reduction using prediction and aggregation techniques in IoT-based wireless sensor networks," *J. Netw. Comput. Appl.*, vol. 211, 2023, Art. no. 103556.

[19] T. Lu, W. Xia, X. Zou, and Q. Xia, "Adaptively compressing {IoT} data on the resource-constrained edge," in *Proc. 3rd USENIX Workshop Hot Topics Edge Comput.*, 2020, pp. 1–7.

[20] A. K. Idrees and M. S. Khlief, "Efficient compression technique for reducing transmitted EEG data without loss in IoMT networks based on fog computing," *J. Supercomput.*, vol. 79, no. 8, pp. 9047–9072, 2023.

[21] A. A. Abdellatif, A. Emam, C. F. Chiasserini, and M. Youssef, "Edge-based compression and classification for smart healthcare systems: Concept, implementation and evaluation," *Expert Syst. Appl.*, vol. 117, pp. 1–14, 2019.

[22] D. Zhang et al., "Understanding taxi service strategies from taxi GPS traces," *IEEE Trans. Intell. Transp. Syst.*, vol. 16, no. 1, pp. 123–135, Feb. 2015.

[23] W. Xia et al., "A comprehensive study of the past, present, and future of data deduplication," *Proc. IEEE*, vol. 104, no. 9, pp. 1681–1710, Sep. 2016.

[24] Y. Cui et al., "QuickSync: Improving synchronization efficiency for mobile cloud storage services," in *Proc. 21st Annu. Int. Conf. Mobile Comput. Netw.*, 2015, pp. 592–603.

[25] S. Wu, L. Liu, H. Jiang, D. Wang, and Y. Zhou, "PandaSync: Network and workload aware hybrid cloud sync optimization," in *Proc. IEEE 39th Int. Conf. Distrib. Comput. Syst.*, 2019, pp. 282–292.

[26] Y. He et al., "DSYNC: A lightweight delta synchronization approach for cloud storage services," in *Proc. 35th Symp. Mass Storage Syst. Technol.*, 2020, pp. 1–14.

[27] S. Wu et al., "FASTSync: A FAST delta sync scheme for encrypted cloud storage in high-bandwidth network environments," *ACM Trans. Storage*, vol. 19, no. 4, pp. 1–22, 2023.

[28] L. Liu, J. Li, Z. Niu, H. Zhang, and Y. Zhang, "Efficient time-series data delivery in IoT with Xender," *IEEE Trans. Mobile Comput.*, vol. 23, no. 5, pp. 4777–4792, May 2024.

[29] J. Yoon, D. Jarrett, and M. Van der Schaar, "Time-series generative adversarial networks," in *Proc. Adv. Neural Inf. Process. Syst.*, 2019, vol. 32.

[30] G. M. Dias, B. Bellalta, and S. Oechsner, "A survey about prediction-based data reduction in wireless sensor networks," *ACM Comput. Surv.*, vol. 49, no. 3, pp. 1–35, 2016.

[31] K. L. Ketshabetswe, A. M. Zungeru, B. Mtengi, C. K. Lebekwe, and S. Prabaharan, "Data compression algorithms for wireless sensor networks: A review and comparison," *IEEE Access*, vol. 9, pp. 136872–136891, 2021.

[32] S. Hamdan, A. Awaian, and S. Almajali, "Compression techniques used in IoT: A comparitive study," in *Proc. 2nd Int. Conf. New Trends Comput. Sci.*, 2019, pp. 1–5.

[33] A. Moon, J. Kim, J. Zhang, and S. W. Son, "Lossy compression on IoT Big Data by exploiting spatiotemporal correlation," in *Proc. IEEE High Perform. Extreme Comput. Conf.*, 2017, pp. 1–7.

[34] J. D. A. Correa, A. S. R. Pinto, and C. Montez, "Lossy data compression for IoT sensors: A review," *Internet Things*, vol. 19, 2022, Art. no. 100516.

[35] K. Hossain, M. Rahman, and S. Roy, "IoT data compression and optimization techniques in cloud storage: Current prospects and future directions," *Int. J. Cloud Appl. Comput.*, vol. 9, no. 2, pp. 43–59, 2019.

[36] H. Harb, A. Makhoul, S. Tawbi, and R. Couturier, "Comparison of different data aggregation techniques in distributed sensor networks," *IEEE Access*, vol. 5, pp. 4250–4263, 2017.

[37] T. Srisooksai, K. Keamarungsi, P. Lamsrichan, and K. Araki, "Practical data compression in wireless sensor networks: A survey," *J. Netw. Comput. Appl.*, vol. 35, no. 1, pp. 37–59, 2012.

[38] M. A. Razzaque, C. Bleakley, and S. Dobson, "Compression in wireless sensor networks: A survey and comparative evaluation," *ACM Trans. Sensor Netw.*, vol. 10, no. 1, pp. 1–44, 2013.

[39] V. K. Singh, V. K. Singh, and M. Kumar, "In-network data processing based on compressed sensing in WSN: A survey," *J. Commun. Inf. Netw.*, vol. 2, no. 4, pp. 1–16, 2017.

[40] B. A. Lungisani, C. K. Lebekwe, A. M. Zungeru, and A. Yahya, "Image compression techniques in wireless sensor networks: A survey and comparison," *IEEE Access*, vol. 10, pp. 82511–82530, 2022.

[41] K. Jain and S. Mohapatra, "Taxonomy of edge computing: Challenges, opportunities, and data reduction methods," in *Edge Computing: From Hype to Reality*. Cham, Switzerland: Springer, 2019, pp. 51–69.

[42] A. A. Sadri and F. Khani, "Data reduction in fog and cloud computing using prediction approaches: A comprehensive review," *Working Paper, SSRN Electron. J.*, Jul. 2024. [Online]. Available: https://ssrn.com/abstract=4907156

[43] D. Kreković, P. Krivić, I. P. Žarko, M. Kušek, and D. Le-Phuoc, "Reducing communication overhead in the IoT-edge-cloud continuum: A survey on protocols and data reduction strategies," *Internet Things*, vol. 31, 2025, Art. no. 101553.

[44] N. Xie and H. Leung, "Internet of Things (IoT) in canadian smart cities: An overview," *IEEE Instrum. Meas. Mag.*, vol. 24, no. 3, pp. 68–77, May 2021.

[45] S.-C. Lo, M.-H. Chiang, J.-H. Liou, and J.-S. Gao, "Routing and buffering strategies in delay-tolerant networks: Survey and evaluation," in *Proc. 40th Int. Conf. Parallel Process. Workshops*, 2011, pp. 91–100.

[46] P. T. Agarkar, M. D. Chawan, P. T. Karule, and P. R. Hajare, "A comprehensive survey on routing schemes and challenges in wireless sensor networks (WSN)," *Int. J. Comput. Netw. Appl.*, vol. 7, no. 6, pp. 193–207, 2020.

[47] B. Yang and J. Leon., "A study of the factors influencing the sales of intelligent connected cars," in *Proc. 4th Int. Conf. Manage. Sci. Eng. Manage.*, 2023, pp. 672–687.

[48] A. Tridgell and P. Mackerras, "The RSYNC algorithm," Tech. Rep. TR-CS-96-05, 1996.

[49] A. Muthitacharoen, B. Chen, and D. Mazieres, "A low-bandwidth network file system," in *Proc. 18th ACM Symp. Operating Syst. Princ.*, 2001, pp. 174–187.

[50] Z. Li et al., "Efficient batched synchronization in dropbox-like cloud storage services," in *Proc. Middleware 2013: ACM/IFIP/USENIX 14th Int. Middleware Conf.*, Beijing, China , 2013, pp. 307–327.

[51] C. Chen, Y. Ding, Z. Wang, J. Zhao, B. Guo, and D. Zhang, "VTracer: When online vehicle trajectory compression meets mobile edge computing," *IEEE Syst. J.*, vol. 14, no. 2, pp. 1635–1646, Jun. 2020.

[52] Y. Wu, B. Yang, C. Li, Q. Liu, Y. Liu, and D. Zhu, "A deep learning based efficient data transmission for industrial cloud-edge collaboration," in *Proc. IEEE 31st Int. Symp. Ind. Electron.*, 2022, pp. 1202–1207.

[53] Y. Wu et al., "To transmit or predict: An efficient industrial data transmission scheme with deep learning and cloud-edge collaboration," *IEEE Trans. Ind. Informat.*, vol. 19, no. 11, pp. 11322–11332, Nov. 2023.

[54] X. Wang, B. Yang, Z. Wang, H. Li, and Y. Liu, "A compressed sensing and CNN-based method for fault diagnosis of photovoltaic inverters in edge computing scenarios," *IET Renewable Power Gener.*, vol. 16, no. 7, pp. 1434–1444, 2022.

[55] Y. Mobarak, S. Mosaad, and A. Kotb, "Split-compression framework for traffic reduction between IoT and cloud," in *Proc. Int. Mobile, Intell., Ubiquitous Comput. Conf.*, 2024, pp. 180–184.

[56] X. Zeng and S. Zhang, "CStream: Parallel data stream compression on multicore edge devices," *IEEE Trans. Knowl. Data Eng.*, vol. 36, no. 11, pp. 5889–5904, Nov. 2024.

[57] C. Liu, J. Paparrizos, and A. J. Elmore, "AdaEdge: A dynamic compression selection framework for resource constrained devices," in *Proc. IEEE 40th Int. Conf. Data Eng.*, 2024, pp. 1506–1519.

[58] J. Acharya and S. Gaur, "Edge compression of GPS data for mobile IoT," in *Proc. IEEE Fog World Congr.*, 2017, pp. 1–6.

[59] V. W. Håkansson, N. K. D. Venkategowda, F. A. Kraemer, and S. Werner, "Cost-aware dual prediction scheme for reducing transmissions at iot sensor nodes," in *Proc. 27th Eur. Signal Process. Conf.*, 2019, pp. 1–5.

[60] Y. Zhou, S. Wu, S. Wang, H. Jiang, and D. Wang, "LearnedSync: A learning-based sync optimization for cloud storage," in *Proc. Int. Conf. Algorithms Architectures Parallel Process.*, 2023, pp. 1–21.

[61] R. K. Barik, H. Dubey, A. B. Samaddar, A. Dubey, G. N. Purohit, and R. Buyya, "FogGIS: Fog computing for geospatial Big Data analytics," in *Proc. IEEE Uttar Pradesh Sect. Int. Conf. Elect., Comput. Electron. Eng.*, 2016, pp. 613–618.

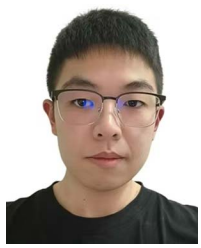
[62] D. Blalock, S. Madden, and J. Guttag, "SprintZ: Time series compression for the Internet of Things," *Proc. ACM Interact., Mobile, Wearable Ubiquitous Technol.*, vol. 2, no. 3, pp. 1–23, 2018.

[63] Z. Hu, D. Wang, Z. Li, Y. Zhang, and W. Wang, "Differential compression for mobile edge computing in internet of vehicles," in *Proc. Int. Conf. Wireless Mobile Comput., Netw. Commun.*, 2019, pp. 336–341.

[64] S. K. Idrees and A. K. Idrees, "New fog computing enabled lossless EEG data compression scheme in IoT networks," *J. Ambient Intell. Humanized Comput.*, vol. 13, no. 6, pp. 3257–3270, 2022.

[65] T. Melissaris, K. Shaw, and M. Martonosi, "Optimizing IoT and web traffic using selective edge compression," 2020, *arXiv:2012.14968*.
[66] W. -Q. Ng, B. -Y. Ooi, X. -Y. Kh'ng, S. -Y. Liew, and E. Symeonaki, "A context-aware data reduction framework for high velocity data," in *Proc. 3rd Int. Conf. Artif. Intell. Data Sci.*, 2022, pp. 158–163.
[67] W. Q. Ng et al., "A users' behavior-aware data reduction framework for Internet of Things (IoT)," in *Proc. Int. Conf. Consum. Electron.-Taiwan*, 2024, pp. 239–240.
[68] Q. Zhang et al., "DeltaCFS: Boosting delta sync for cloud storage services by learning from NFS," in *Proc. IEEE 37th Int. Conf. Distrib. Comput. Syst.*, 2017, pp. 264–275.
[69] H. Xiao et al., "Towards web-based delta synchronization for cloud storage services," in *Proc. 16th USENIX Conf. File Storage Technol.*, 2018, pp. 155–168.
[70] C. Zhang et al., "SimpleSync: A parallel delta synchronization method based on flink," *Concurrency Comput.: Pract. Exp.*, vol. 33, no. 20, 2021, Art. no. e6327.
[71] S. Wu, Z. Tu, Z. Wang, H. Jiang, and D. Wang, "When delta sync meets message-locked encryption: A feature-based delta sync scheme for encrypted cloud storage," in *Proc. IEEE 41st Int. Conf. Distrib. Comput. Syst.*, 2021, pp. 337–347.
[72] J. Zheng et al., "Webassembly-based delta sync for cloud storage services," *ACM Trans. Storage*, vol. 18, no. 3, pp. 1–31, 2022.
[73] W. Xia, C. Wei, Z. Li, X. Wang, and X. Zou, "NetSync: A network adaptive and deduplication-inspired delta synchronization approach for cloud storage services," *IEEE Trans. Parallel Distrib. Syst.*, vol. 33, no. 10, pp. 2554–2570, Oct. 2022.
[74] W. Zhang et al., "SPSYNC: Lightweight multi-terminal big spatiotemporal data synchronization solution," *Future Gener. Comput. Syst.*, vol. 141, pp. 106–115, 2023.
[75] R. Pan, Y. Liang, L. Li, H. Du, T.-W. Kuo, and C. J. Xue, "Efficient hash-free mobile cloud backup via operation-log versioning," *ACM Trans. Storage*, early access, Jan. 2026, doi: 10.1145/3789213.
[76] Z. Zhang et al., "{ParaSync }: Exploiting { Fine-Grained} parallelism for efficient file synchronization," in *Proc. 24th USENIX Conf. File Storage Technol.*, 2026, pp. 477–492.
[77] J. Yang et al., "{ShieldReduce }:{ Fine-Grained} shielded data reduction," in *Proc. USENIX Annu. Tech. Conf.*, 2025, pp. 1281–1296.
[78] F. Chen, Z. Li, C. Jiang, T. Xiang, and Y. Yang, "Cloud object storage synchronization: Design, analysis, and implementation," *IEEE Trans. Parallel Distrib. Syst.*, vol. 33, no. 12, pp. 4295–4310, Dec. 2022.
[79] A. Papageorgiou, B. Cheng, and E. Kovacs, "Real-time data reduction at the network edge of Internet-of-Things systems," in *Proc. 11th Int. Conf. Netw. Serv. Manage.*, 2015, pp. 284–291.
[80] S. Kadhum Idrees, J. Azar, R. Couturier, A. Kadhum Idrees, and F. Gechter, "SZ4IoT: An adaptive lightweight lossy compression algorithm for diverse IoT devices and data types," *J. Supercomput.*, vol. 81, no. 2, 2025, Art. no. 392.
[81] Y. Wu, B. Yang, D. Zhu, and C. Chen, "LI-DPS: A long sequence dual prediction scheme based on informer for efficient high-frequency data transmission," in *Proc. 50th Annu. Conf. IEEE Ind. Electron. Soc.*, 2024, pp. 1–6.
[82] A. A. Sadri, A. M. Rahmani, M. Saberikamarposhti, and M. Hosseinzadeh, "A predictive scheme in fog computing to reduce the volume of data sent to cloud layer," *Cluster Comput.*, vol. 29, no. 3, 2026, Art. no. 194.
[83] D. Kreković, M. Kušek, I. P. Žarko, and D. Le-Phuoc, "Edge-based predictive data reduction for smart agriculture: A lightweight approach to efficient IoT communication," in *Proc. IEEE Annu. Congr. Artif. Intell. Things*, 2025, pp. 382–389.
[84] S. Lu, Q. Xia, X. Tang, X. Zhang, Y. Lu, and J. She, "A reliable data compression scheme in sensor-cloud systems based on edge computing," *IEEE Access*, vol. 9, pp. 49007–49015, 2021.
[85] A. Andrade, C. A. da Costa, A. Roehrs, D. Muchaluat-Saade, and R. da Rosa Righi, "Blending lossy and lossless data compression methods to support health data streaming in smart cities," *Future Gener. Comput. Syst.*, vol. 167, 2025, Art. no. 107748.
[86] L. Pioli, D. D. de Macedo, D. G. Costa, and M. A. Dantas, "Intelligent edge-powered data reduction: A systematic literature review," *ACM Comput. Surv.*, vol. 56, no. 9, pp. 1–39, 2024.
[87] R. F. Alwash, A. K. Idrees, and S. Al-Obaidi, "A methodological review on EEG data reduction in EDGE/FOG computing-based IoMT networks," in *Proc. 14th Int. Conf. Comput. Commun. Netw. Technol.*, 2023, pp. 1–7.
[88] C. Reichenberger, "Delta storage for arbitrary non-text files," in *Proc. 3rd Int. Workshop Softw. Configuration Manage.*, 1991, pp. 144–152.
[89] R. C. Agarwal, K. Gupta, S. Jain, and S. Amalapurapu, "An approximation to the greedy algorithm for differential compression," *IBM J. Res. Dev.*, vol. 50, no. 1, pp. 149–166, 2006.
[90] W. Xia et al., "FastCDC: A fast and efficient content-defined Chunking approach for data deduplication," in *Proc. USENIX Annu. Tech. Conf.*, 2016, pp. 101–114.
[91] "Googledrive," (n.d.). Accessed: May 2026. [Online]. Available: https://www.microsoft.com/en-us/microsoft-365/onedrive/online-cloud-storage
[92] "Onedrive," (n.d.). Accessed: May 2026. [Online]. Available: https://www.dropbox.com
[93] R. Song, W. Sun, B. Zheng, and Y. Zheng, "PRESS: A novel framework of trajectory compression in road networks," in *Proc. VLDB Endowment*, vol. 7, no. 9, 2014, pp. 661–672.
[94] Y. Ji, Y. Zang, W. Luo, X. Zhou, Y. Ding, and L. M. Ni, "Clockwise compression for trajectory data under road network constraints," in *Proc. 2016 IEEE Int. Conf. Big Data (Big Data)*, 2016, pp. 472–481.
[95] Y. Fathy, P. Barnaghi, and R. Tafazolli, "An adaptive method for data reduction in the Internet of Things," in *Proc. IEEE 4th World Forum Internet of Things*, 2018, pp. 729–735.
[96] H. Zhou et al., "Informer: Beyond efficient transformer for long sequence time-series forecasting," in *Proc. AAAI Conf. Artif. Intell.*, 2021, vol. 35, no. 12, pp. 11106–11115.
[97] J. Liu, K. Wang, and F. Chen, "Understanding energy efficiency of databases on single board computers for edge computing," in *Proc. 29th Int. Symp. Model., Anal., Simul. Comput. Telecommun. Syst.*, 2021, pp. 1–8.
[98] T. Pelkonen et al., "Gorilla: A fast, scalable, in-memory time series database," *Proc. VLDB Endowment*, vol. 8, no. 12, pp. 1816–1827, 2015.
[99] P. Liakos, K. Papakonstantinopoulou, and Y. Kotidis, "CHIMP: Efficient lossless floating point compression for time series databases," *Proc. VLDB Endowment*, vol. 15, no. 11, pp. 3058–3070, 2022.
[100] R. Li, Z. Li, Y. Wu, C. Chen, and Y. Zheng, "ELF: Erasing-based lossless floating-point compression," *Proc. VLDB Endowment*, vol. 16, no. 7, pp. 1763–1776, 2023.

Jian Liu received the PhD degree in computer science from Louisiana State University, USA, in 2021. He is currently an assistant professor with the College of Computer Science and Technology, Zhejiang University of Technology, China. His research interests include Big Data management, storage/cache systems, machine learning, and edge computing.



Yangyang Lin received the bachelor's degree in software engineering from the Zhejiang University of Technology. He is currently working toward the master's degree with the College of Computer Science and Technology, Zhejiang University of Technology. His research interests include deep learning, Big Data management, and edge computing.

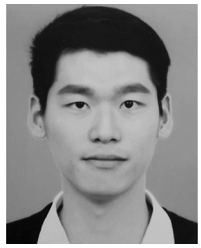
Ziguang Fu received the bachelor's degree in software engineering from the Zhejiang University of Technology. He is currently working toward the master's degree in computer science and technology with the Zhejiang University of Technology. His research interests include time-series data processing and IoT technology.

Gexi Lin is currently working toward the undergraduation degree with the College of Computer Science and Technology, Zhejiang University of Technology. Her research interests include Big Data management and edge computing.

Guodao Sun received the BSc degree in computer science and technology and the PhD degree in control science and engineering from the Zhejiang University of Technology, Hangzhou, China. He is currently a professor with the College of Computer Science and Technology, Zhejiang University of Technology. His research interests include the area of visual analytics and artificial intelligence.

Zhu Xiao (Senior Member, IEEE) received the MS and PhD degrees in communication and information systems from Xidian University, Xi'an, China, in 2007 and 2009, respectively. From 2010 to 2012, he was a research fellow with the Department of Computer Science and Technology, University of Bedfordshire, Bedfordshire, U.K. He is currently a full professor with the College of Computer Science and Electronic Engineering, Hunan University, Changsha, China. His research interests include machine learning, mobile computing, and intelligent information processing. He has actively served on many conference committees. He is serving as an associate editor for *IEEE Transactions on Intelligent Transportation Systems*.

Yilong Zhang (Member, IEEE) received the PhD degree from Tsinghua University, China, in 2017. He is currently an associate professor with the College of Computer Science and Technology, Zhejiang University of Technology. His research interests include computational imaging, biomedical optical imaging, and 3D shape measurement and reconstruction.



Dongdong Zhao received the BSc degree from Shandong University, in 2013 and the PhD degree from Zhejiang University, Hangzhou, China, in 2019. He is currently a lecturer with the Zhejiang University of Technology. His research interests include signal processing, parallel computing, and embedded system.

Peng Chen (Member, IEEE) received the BSc and PhD degrees from Zhejiang University, Hangzhou, China, in 2003 and 2009 respectively. From 2015 to 2016, he was a visiting scholar with the University of California—Santa Barbara, Santa Barbara, CA, USA. He is currently a professor with the Zhejiang University of Technology, Hangzhou. His research interests include computer vision, embedded systems design, and pattern recognition.

Ronghua Liang (Senior Member, IEEE) received the PhD degree in computer science from Zhejiang University, in 2003. He is currently a professor of computer science and dean of the College of Computer Science and Technology, Zhejiang University of Technology, China. He has authored or coauthored more than 50 papers in leading international journals and conferences, such as *IEEE Transactions on Knowledge and Data Engineering*, *IEEE Transactions on Visualization and Computer Graphics*, IEEE VIS, IJCAI, and AAAI.